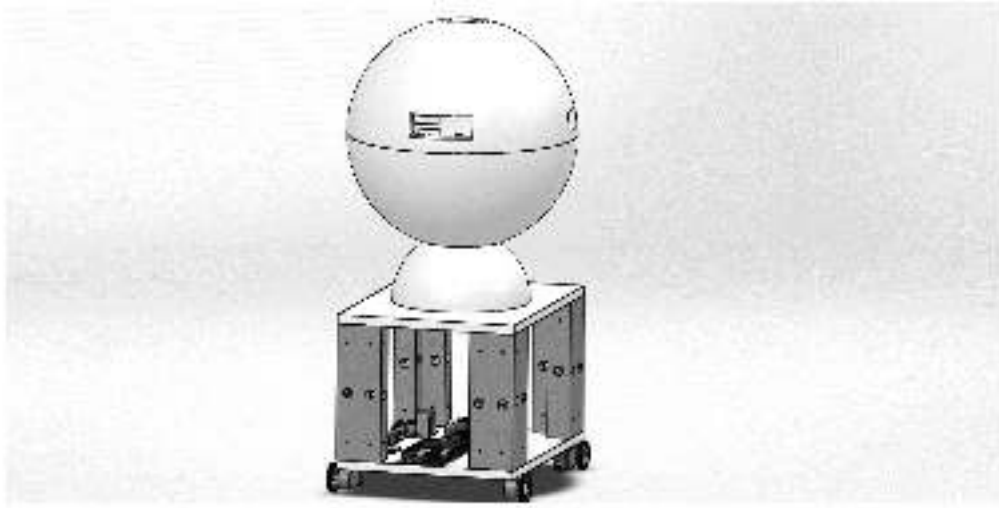


THIRD REVISED AND EXPANDED EDITION

Robotinics Rev. 3

Robotics, IoT, and Artificial Intelligence with Arduino, Raspberry Pi,
and TCHATGPT



Marcelo Maurin Martins

Third revised and expanded edition • 2026

EDITION INFORMATION

Robotinics Rev. 3

Robotics, IoT, and Artificial Intelligence with Arduino, Raspberry Pi, and TCHATGPT

Author and project lead: Marcelo Maurin Martins

Third revised and expanded edition, 2026.

Language: English (United States).

Project	Editorial snapshot
Robotinics	61d2a19d10e6701e27e46181e1a6c740b8ee4e30
TCHATGPT	15d5e1a7780088701716896cfe9fb3afe0e7b71a

WARNING Educational and experimental project. Lithium-ion batteries, tools, soldering, motors, and moving parts require suitable protection, supervision, and components. AI never replaces deterministic safety barriers.

Project sources: [Robotinics repository](#) and [TCHATGPT repository](#).

Editorial note: the examined Robotinics snapshot contains no formal license. Public release of this edition should be accompanied by the license chosen by the author.

To my beloved wife, Daniela Machado.

Eternal love.

Contents

Click an entry to navigate within the document.

<u>Introduction to the third edition</u>	6
<u>What was corrected</u>	6
<u>How to use this book</u>	7
<u>Conventions</u>	7
<u>Acknowledgments</u>	8
<u>Part I - Project, architecture, and safety</u>	9
<u>1. Robotinics as a learning platform</u>	10
<u>1.1 Robot, IoT, and cyber-physical system</u>	10
<u>1.2 Rev. 3 objectives</u>	10
<u>1.3 Intended audience</u>	10
<u>1.4 Repository organization</u>	10
<u>1.5 Documentation strategy</u>	11
<u>2. Third-edition architecture</u>	12
<u>2.1 Separation of responsibilities</u>	12
<u>2.2 Command path</u>	12
<u>2.3 Deployment profiles</u>	13
<u>2.4 Operating states</u>	14
<u>2.5 Interfaces</u>	14
<u>3. Safety and working method</u>	15
<u>3.1 Safety hierarchy</u>	15
<u>3.2 Bench rules</u>	15
<u>3.3 Change control</u>	15
<u>3.4 Stage completion criteria</u>	16
<u>3.5 Credential and data security</u>	16
<u>3.6 Stated limitations</u>	16
<u>Part II - Mechanics, power, and electronics</u>	17
<u>4. Mechanical sizing</u>	18
<u>4.1 Requirements before selecting motors</u>	18
<u>4.2 Motion forces</u>	18
<u>4.3 Corrected traction example</u>	18
<u>4.4 Grip and traction limit</u>	19
<u>4.5 Arm torque</u>	19
<u>4.6 Center of gravity and stability</u>	19
<u>5. Structure and mechanical assembly</u>	20
<u>5.1 Mechanical archive</u>	20
<u>5.2 Material and printing</u>	20
<u>5.3 Base</u>	20
<u>5.4 Body, arms, and head</u>	21
<u>5.5 Alternatives to 3D printing</u>	21
<u>5.6 Mechanical acceptance criteria</u>	22
<u>6. Power, battery, and distribution</u>	23
<u>6.1 Fundamental quantities</u>	23
<u>6.2 3S pack</u>	23
<u>6.3 Safe charging</u>	23
<u>6.4 BMS, fuse, and master switch</u>	23
<u>6.5 Separate rails</u>	24
<u>6.6 Converters</u>	25
<u>6.7 Runtime</u>	25
<u>6.8 Capacitors and ripple</u>	25
<u>6.9 Correct measurement</u>	25
<u>6.10 Electrical acceptance criteria</u>	25
<u>7. Electronics, sensors, and actuators</u>	27
<u>7.1 Arduino and logic levels</u>	27
<u>7.2 Motor driver</u>	27
<u>7.3 Relays</u>	27
<u>7.4 LEDs</u>	27

7.5 Ultrasonic sensor.....	27
7.6 Servos.....	28
7.7 Analog sensors.....	28
7.8 Decoupling and electromagnetic compatibility.....	28
7.9 Electronics checklist.....	28
Part III - Firmware, Linux, and communication.....	30
8. Robotinics Rev. 3 firmware.....	31
8.1 Why reorganize it.....	31
8.2 Modules.....	31
8.3 Safe initialization.....	31
8.4 Bounded serial input.....	32
8.5 Command manual.....	32
8.6 Watchdog and heartbeat.....	32
8.7 Obstacles and local limits.....	33
8.8 Telemetry.....	33
8.9 Avoiding memory fragmentation.....	33
8.10 Firmware tests.....	33
9. Raspberry Pi and robot services.....	34
9.1 Role of Raspberry Pi.....	34
9.2 System installation.....	34
9.3 Minimum profile.....	34
9.4 Services with systemd.....	34
9.5 Dependencies and installer.....	35
9.6 Camera and audio.....	35
9.7 Storage.....	35
9.8 Update and rollback.....	35
10. Communication, data, and software security.....	36
10.1 Serial before networking.....	36
10.2 Historical servers.....	36
10.3 Ports and contracts.....	36
10.4 Network interface.....	36
10.5 Structured messages.....	36
10.6 Database.....	37
10.7 Observability.....	37
10.8 Resilience tests.....	37
Part IV - Artificial intelligence with TCHATGPT.....	39
11. Preparing TCHATGPT.....	40
11.1 The project.....	40
11.2 Providers.....	40
11.3 Configuration without secrets in code.....	41
11.4 Asynchronous calls.....	41
11.5 Prompts and contracts.....	41
11.6 Self-test.....	41
12. Supervised control agent.....	43
12.1 Components.....	43
12.2 Safe configuration.....	43
12.3 Capability registration.....	43
12.4 Event before the action.....	44
12.5 Understandable confirmation.....	44
12.6 Deterministic discovery.....	44
12.7 Session memory.....	45
12.8 Current limits.....	45
12.9 Adversarial tests.....	45
13. Voice and human interaction.....	46
13.1 Pipeline.....	46
13.2 Speech recognition.....	46
13.3 Voice is not authorization.....	46
13.4 Traditional synthesis and voice cloning.....	46
13.5 Privacy.....	47
13.6 Acceptance criteria.....	47
14. Computer vision.....	48
14.1 Historical correction.....	48

14.2 Current components.....	48
14.3 Robotinics pipeline.....	48
14.4 Performance on Raspberry Pi.....	49
14.5 Safety and ethics.....	49
14.6 Tests.....	49
15. Memory and RAG.....	50
15.1 Use in the robot.....	50
15.2 TAIRAG.....	50
15.3 Separation of knowledge and authority.....	50
15.4 Sources and traceability.....	50
15.5 Chunking.....	51
15.6 Operational memory.....	51
15.7 Evaluation.....	51
16. Intelligent behaviors and limits.....	52
16.1 Operating modes.....	52
16.2 Appropriate capabilities.....	52
16.3 Plan, execute, and verify.....	52
16.4 Uncertainty.....	53
16.5 AI observability.....	53
16.6 Promotion criterion.....	53
Part V - Construction, validation, and evolution.....	54
17. Construction sequence.....	55
17.1 Stage 0 - documentation and inventory.....	55
17.2 Stage 1 - passive mechanics.....	55
17.3 Stage 2 - power distribution.....	55
17.4 Stage 3 - Arduino without actuators.....	55
17.5 Stage 4 - raised traction system.....	55
17.6 Stage 5 - servos and sensors.....	56
17.7 Stage 6 - Raspberry Pi and gateway.....	56
17.8 Stage 7 - TCHATGPT in simulation.....	56
17.9 Stage 8 - agent with hardware.....	56
17.10 Stage 9 - voice, vision, and RAG.....	56
17.11 First floor operation.....	57
18. Test and commissioning plan.....	58
18.1 Test pyramid.....	58
18.2 Minimum matrix.....	58
18.3 Electrical measurements.....	59
18.4 Mechanical measurements.....	59
18.5 AI evaluation.....	59
18.6 Fault injection.....	60
18.7 Test record.....	60
18.8 Commissioning criterion.....	60
19. Maintenance and evolution.....	61
19.1 Periodic inspection.....	61
19.2 Compatibility.....	61
19.3 Reproducible repository.....	61
19.4 Technical roadmap.....	61
19.5 Publication.....	62
Appendix A - Suggested bill of materials.....	63
Appendix B - Command protocol.....	64
B.1 Rules.....	64
B.2 Minimum catalog.....	64
B.3 Legacy aliases.....	64
Appendix C - Correction matrix for the previous edition.....	65
Appendix D - Reference formulas.....	67
Appendix E - Platform and dependency matrix.....	68
Appendix F - References.....	69
Projects.....	8
Official technical documentation.....	69
Appendix G - Glossary.....	70
Closing remarks.....	71

Introduction to the third edition

Robotinics began as a multidisciplinary project: a ground robot open to experimentation, built with 3D-printed parts, accessible electronics, Arduino, Raspberry Pi, and free and open-source software. The first edition documented a real journey of research and construction. This third edition preserves that historical value while reorganizing the material so that it once again works as a technical manual.

The main change is not merely an update to software versions. The architecture was revised to separate responsibilities, correct calculations, and incorporate artificial intelligence without giving a language model unrestricted control of the robot. Arduino remains responsible for deterministic and real-time tasks. Raspberry Pi coordinates services, sensors, and communication. The TCHATGPT project provides the layer for language models, agents, voice, vision, memory, RAG, and observability. A mandatory validation and safety layer sits between every AI decision and any actuator.

This edition was built on two frozen references:

Project	Editorial reference	Purpose
Robotinics	commit 61d2a19d10e6701e27e46181e1a6c740b8ee4e30	Mechanics, electronics, firmware, and historical archive
TCHATGPT	commit 15d5e1a7780088701716896cfe9fb3afe0e7b71a	Lazarus/Free Pascal components for AI and integration

Freezing these references does not prevent future development. It only ensures that readers can relate the text to the code. Newer versions may be adopted after their differences have been verified.

WARNING This is an educational and experimental project. It includes motors, lithium-ion batteries, tools, soldering, and moving parts. Do not work on an energized system, do not assemble improvised lithium chargers, and do not make mains-voltage connections based solely on this book. Use certified modules, adequate protection, and supervision by a qualified person.

What was corrected

Corrections in this edition include, among others:

- series-cell capacity and the nominal and maximum voltage of a 3S pack;
- the CC/CV charging process, protection, and balancing of lithium cells;
- correct multimeter use for voltage and current measurements;
- the distinction between force, torque, power, energy, and capacity;
- traction, slope, and arm-torque calculations;
- LED current and sizing of the current-limiting resistor;
- definitions of normally open and normally closed relay contacts;
- actual buck-converter limits and separation of motor, servo, and logic power rails;
- the L298N description, PWM speed control, and component limitations;
- HC-SR04 frequency, code examples, and timing requirements;

- safety and memory problems in the historical C/C++ servers;
- conflicting ports and commands, and files absent from the current repository;
- outdated Linux, database, PHP, OpenCV, voice, and remote-access practices;
- navigation, references, terminology, and editorial organization.

A detailed matrix appears in Appendix C.

How to use this book

The project is divided into five parts. Part I defines requirements, architecture, and safety. Part II covers mechanics, power, and electronics. Part III presents firmware, Linux, and communication. Part IV integrates TCHATGPT. Part V guides assembly, testing, and evolution.

Readers do not need to install every tool to build the robot. Those who only want to reproduce the prototype can use the ready-made files. Those who intend to modify parts, boards, or software should work from the version-controlled sources. In both cases, every stage ends with acceptance criteria: do not proceed until the current subsystem is stable.

Conventions

- Voltage values are given in volts (V), current in amperes (A), power in watts (W), energy in watt-hours (Wh), force in newtons (N), and torque in newton-meters (N·m).
- Numerical examples are instructional. When a real component has a different datasheet, the datasheet takes precedence.
- Literal commands, filenames, and identifiers appear in monospaced type.
- Sections marked **Experimental** have not been sufficiently validated on the indicated platform.
- The term **AI** includes language models, vision, voice, classification, and agents. It does not imply unrestricted autonomy.

Acknowledgments

This work is the result of years of study, construction, and collaboration. Marcelo Maurin Martins remains grateful to God, his family, friends, professors at Centro Paula Souza, ETEC, and FATEC, and everyone who reviewed or supported the original project. The memory of Professor Danilo continues to be honored in this edition.

The Arduino, Raspberry Pi, Free Pascal, Lazarus, and free and open-source software communities are also recognized; their work makes it possible to build and share projects such as Robotinics.

Part I - Project, architecture, and safety

ROBOTINICS REV. 3

1. Robotinics as a learning platform

1.1 Robot, IoT, and cyber-physical system

A robot is an electromechanical system capable of sensing its environment, processing information, and producing actions. An Internet of Things device collects or receives data and exchanges it with other systems. Robotinics belongs to both groups: it has sensors, actuators, local processing, network communication, and services that can be integrated with external applications.

This definition is useful because it prevents the project from being reduced to a single board. Behavior emerges from cooperation among the mechanical structure, power supply, electronics, firmware, operating system, and applications. A power failure may look like a software error. Motor noise may corrupt a sensor reading. Network delay may make a motion command unsafe. The complete system is therefore the unit of design.

1.2 Rev. 3 objectives

The third edition adopts the following verifiable objectives:

1. Keep the robot controllable when the network or AI is unavailable.
2. Prevent free-form text from being converted directly into a motor signal.
3. Allow deterministic discovery of the commands published by the firmware.
4. Separate traction, servo, and logic power.
5. Record the version, state, and failures of every subsystem.
6. Run staged tests before full assembly.
7. Offer local and remote AI modes without requiring a specific provider.
8. Preserve compatibility with the mechanical archive and historical commands when doing so does not compromise safety.

1.3 Intended audience

This book is intended for students, teachers, makers, and developers who want to understand integration across several fields. It does not assume professional experience in all of them, but it does require care. A beginner can reproduce low-risk stages; activities involving batteries, high current, tools, or structural modifications should be performed with suitable guidance.

For programmers, Robotinics provides a real case study involving embedded systems, protocols, Lazarus, Free Pascal, C/C++, Linux, vision, voice, and AI. For electronics and mechanical professionals, it shows how to transform physical components into an observable and programmable system.

1.4 Repository organization

In the adopted snapshot, the main files are located at:

Area	Reference path	Contents
Mechanics	Mecanic/solidwork and Mecanic/stl	Assemblies, parts, and 3D-printing files
Electronics	Eletronic/eagle , Eletronic/pcb	Boards, drawings, and historical manufacturing files
Arduino	Software/arduino	Main firmware and head firmware
Raspberry	Software/raspberry	Services, camera, voice, and historical applications
Database and web	Software/database and Software/site	Scripts and historical interface
Book	docs	PDF of the previous edition

Some Lazarus tools mentioned in the previous edition were removed from the main branch in 2022. They remain recoverable from the history, but they must not be presented as though they were available in the current clone. Rev. 3 replaces their role with examples based on the modular TCHATGPT packages.

1.5 Documentation strategy

Concepts, calculations, architecture, and test criteria belong in the book. Installation sequences that change frequently should remain in version-controlled repository documentation. This separation reduces obsolescence: the book teaches why and how to validate, while the repository provides the exact command compatible with a given version.

Before starting, record the following on a build sheet:

- mechanical revision of the parts;
- Arduino and Raspberry Pi board models;
- model, chemistry, and declared battery capacity;
- motor and servo models;
- firmware version;
- commits of the Robotinics and TCHATGPT projects;
- operating system and architecture;
- AI provider or local server;
- bench-test results.

2. Third-edition architecture

2.1 Separation of responsibilities

The architecture uses layers. Arduino performs low-level control: motors, servos, simple readings, timing, limits, and stopping. Raspberry Pi runs the robot services: communication, capture of high-volume sensor data, supervision, and integration. TCHATGPT interprets intent, organizes context, and coordinates AI components. The human interface presents requests and confirms sensitive actions.

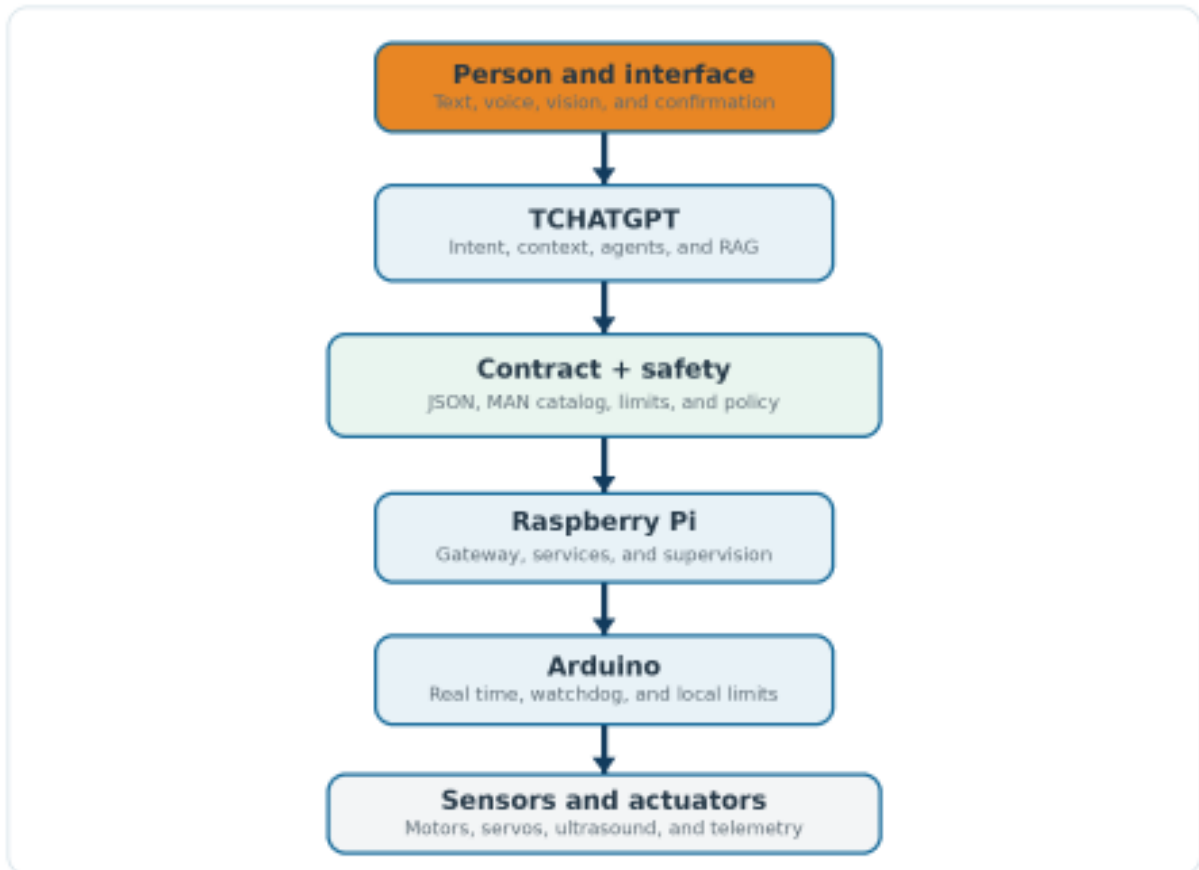


Figure 1 - Layered architecture of the third edition.
Source: Created by the author.

This separation is deliberate. Language models are probabilistic and may produce an incorrect response. Motors require predictable behavior. The model may suggest a structured action; only the validator decides whether the action is in the catalog, whether its parameters are in range, and whether the robot state permits execution.

2.2 Command path

The safe path of a request is:

1. The user speaks or types an intent.
2. The system converts the input to text and includes only the necessary context.
3. The agent asks the model for a response in a structured format.

4. A parser validates the structure; text outside the contract is not executed.
5. The command is compared with the catalog discovered through MAN.
6. The safety policy validates state, parameters, limits, and the need for confirmation.
7. The user confirms physical actions when required.
8. The command is sent to Arduino.
9. The firmware applies its own limits and watchdog.
10. Telemetry confirms the result, and the agent informs the user.

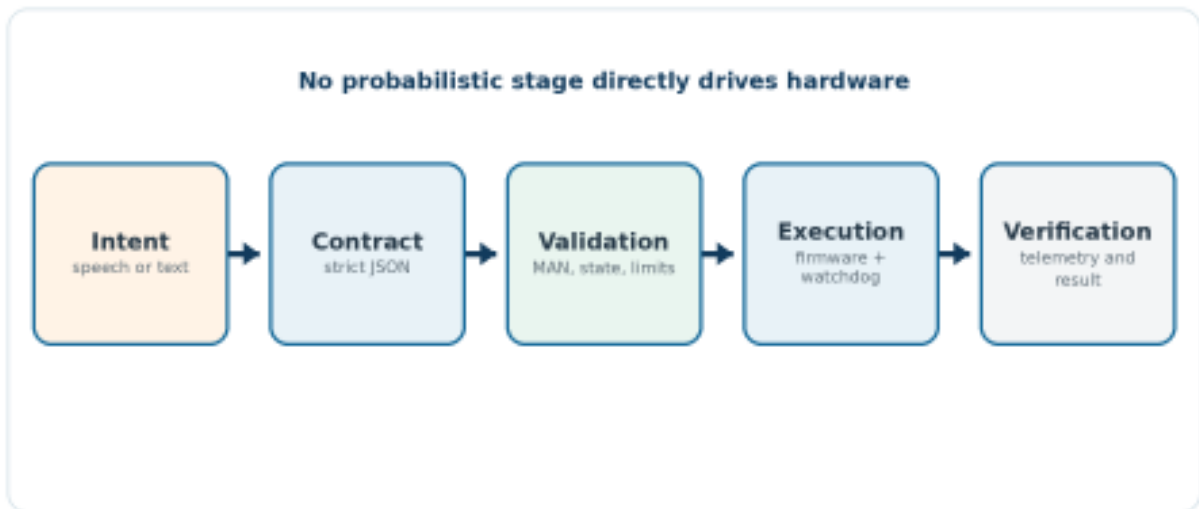


Figure 2 - Supervised path of a physical command.
Source: Created by the author.

The model is never the only barrier. If it invents a command, the catalog blocks it. If it uses an out-of-range parameter, the validator blocks it. If communication stops, the watchdog interrupts motion. If an obstacle is detected, the firmware can stop without consulting the AI.

2.3 Deployment profiles

Profile A - validated controller

The Lazarus application with TCHATGPT runs on Windows x64 or Linux x64. Raspberry Pi acts as the gateway and onboard computer. This is the recommended profile for the first build because these platforms have stronger evidence of support in the TCHATGPT project.

Profile B - AI embedded on Raspberry Pi

The Lazarus application and compatible packages run on Raspberry Pi ARM64. The model may be remote or local. In the snapshot used for this edition, ARM64 is classified as experimental; this profile therefore requires compilation, dependency testing, and validation of every component before controlling actuators.

Profile C - local operation without cloud services

TCHATGPT uses a local OpenAI-compatible endpoint, such as llama.cpp or neural-api. This profile avoids sending robot data to an external provider, but it requires memory, storage, and computing capacity

appropriate for the model. On limited hardware, a small model can handle classification and short commands while vision and voice use specialized components.

2.4 Operating states

The robot uses at least four states:

State	Motion	Query commands	Allowed transition
DISARMED	Blocked	Allowed	To ARMED after verification and confirmation
ARMED	Allowed within limits	Allowed	To EXECUTING, DISARMED, or FAULT
EXECUTING	Current action only	Telemetry allowed	Returns to ARMED or moves to FAULT
FAULT	Blocked with safe output	Diagnostics allowed	To DISARMED after correction and reset

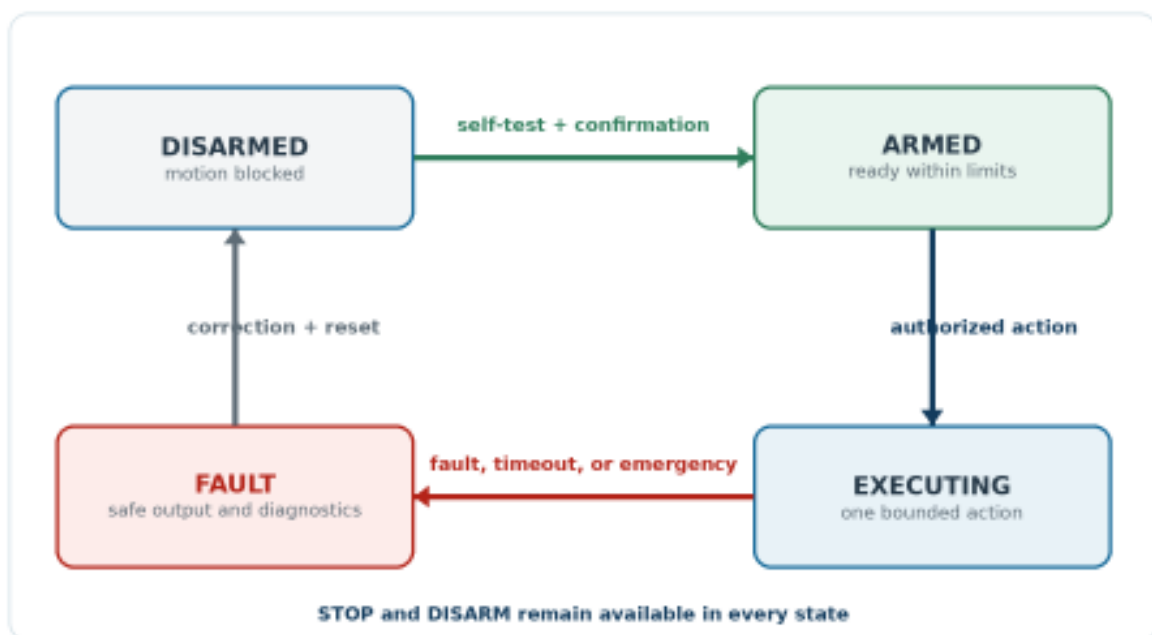


Figure 3 - Robotinics Rev. 3 operating-state machine.

Source: Created by the author.

STOP and DISARM must be accepted in every state. Loss of heartbeat, timeout, critical undervoltage, overcurrent, or activation of the emergency-stop button leads to a safe output without depending on Raspberry Pi.

2.5 Interfaces

The protocol between the controller and Arduino is text-based, line-delimited, and suitable for terminal debugging. A production version may evolve to frames with length, sequence, and integrity checking, but it should retain a human-readable diagnostic mode.

Between network services, prefer authenticated connections, restricted binding, and size-limited messages. Historical servers that accepted commands on every interface and forwarded data to a shell are not part of the recommended architecture.

3. Safety and working method

3.1 Safety hierarchy

Safety is implemented in independent layers:

1. **Mechanical:** gear guards, secure mounting, center of gravity, and no accessible pinch points.
2. **Electrical:** fuse, master switch, reverse-polarity protection, BMS, properly sized wires, and suitable connectors.
3. **Firmware:** limits, timeout, watchdog, obstacle stopping, and disarmed state at startup.
4. **Application:** command catalog, ranges, confirmation, authentication, and logging.
5. **AI:** constrained prompt, structured output, action limits, and no direct operating-system access.
6. **Operation:** clear area, supervision, emergency-stop button, and test procedure.

No layer authorizes removal of the next one. A well-written prompt does not replace a watchdog; a fuse does not replace current limiting in the design; an on-screen confirmation does not replace a physical button.

3.2 Bench rules

- Perform initial tests with the wheels raised off the surface.
- Use a current-limited bench supply before using the battery.
- Connect one subsystem at a time.
- Never adjust wiring while the system is energized.
- Keep the emergency-stop button within reach.
- Test **STOP** before any motion command.
- Remove arms and loads from servos during initial movements.
- Do not charge lithium cells inside the robot without suitable thermal design and protection.
- Do not leave the pack charging unattended.
- Record idle current, peak current, and temperature.

3.3 Change control

Every change should answer four questions:

1. Which requirement motivated the change?
2. Which files and components were changed?
3. How was the change tested?
4. How can the previous version be restored?

Firmware, application, and documentation should report their versions. A published version of the book should point to a repository tag; pointing only to **main** or **master** makes reproduction dependent on future changes.

3.4 Stage completion criteria

A stage is complete when:

- the assembly matches the drawing or a recorded modification;
- there are no loose wires, visible shorts, or unexpected heating;
- consumption is within the power budget;
- all expected commands respond;
- simulated failures lead to a safe state;
- results have been recorded with date and version;
- the next subsystem can be added without hiding earlier failures.

3.5 Credential and data security

API keys must not be stored in the repository or in a readable `settings.ini`. The TCHATGPT serial-agent example has a historical option for storing a token as plain text; this edition does not adopt it. Use an environment variable, operating-system vault, or protected file outside the project directory.

Camera, microphone, telemetry, and conversation data may contain personal information. When a remote provider is used, inform the operator, send only what is necessary, and apply a retention policy. For educational use, prefer controlled environments and non-identifying data.

3.6 Stated limitations

Robotinics is not a medical device, certified industrial equipment, a vehicle for carrying people, or a safety-critical platform. Use in another context requires a risk analysis, applicable standards, and professional validation. Low-cost components and clones may have different specifications; always measure the actual unit.

Part II - Mechanics, power, and electronics

ROBOTINICS REV. 3

4. Mechanical sizing

4.1 Requirements before selecting motors

A motor should not be selected solely from a commercial torque rating. First define mass, wheel diameter, speed, maximum slope, floor type, desired acceleration, and number of driven wheels. Then calculate the required effort and apply a margin.

Force and torque are not the same quantity. Force is measured in newtons. Torque is the product of force and the perpendicular distance to the axis and is measured in newton-meters. A force of 10 N applied 0.05 m from the axis produces 0.5 N·m.

4.2 Motion forces

On a slope at angle θ , the component of weight opposing motion is:

TEXT

$$F_{\text{slope}} = m \times g \times \sin(\theta)$$

The approximate rolling resistance is:

TEXT

$$F_{\text{rolling}} = C_{rr} \times m \times g \times \cos(\theta)$$

For acceleration:

TEXT

$$F_{\text{acceleration}} = m \times a$$

At low speed, Robotinics aerodynamic drag tends to be small compared with the other components, but it can be included when necessary. The total design force is the sum of the relevant components.

Required wheel torque is:

TEXT

$$T_{\text{total}} = (F_{\text{total}} \times \text{wheel_radius}) / \text{efficiency}$$

Divide the result among the wheels that are actually driven. Efficiency represents transmission, deformation, bearing, and floor-contact losses. Do not assume 100%.

4.3 Corrected traction example

Consider a 12 kg robot with two driven wheels, a radius of 0.05 m, a 5° slope, an estimated rolling coefficient of 0.03, acceleration of 0.2 m/s², and overall efficiency of 70%.

Component	Calculation	Approximate result
Slope	$12 \times 9.81 \times \sin(5^\circ)$	10.26 N
Rolling	$0.03 \times 12 \times 9.81 \times \cos(5^\circ)$	3.52 N
Acceleration	12×0.2	2.40 N
Total	sum	16.18 N

Total axle torque is approximately $(16.18 \times 0.05) / 0.70 = 1.16 \text{ N}\cdot\text{m}$. With two driven wheels, each geared-motor assembly must provide about $0.58 \text{ N}\cdot\text{m}$ under the calculated condition. Applying a safety factor of 2 raises the target to approximately $1.16 \text{ N}\cdot\text{m}$ per wheel, or about $11.8 \text{ kgf}\cdot\text{cm}$.

This value must be compared with the assembly's continuous torque, not only its stall torque. Continuous operation near stall heats the motor, increases consumption, and shortens its service life. Also confirm speed after reduction.

4.4 Grip and traction limit

Even a powerful motor cannot move the robot if the wheel slips. The maximum transferable force depends on the normal load on the driven wheels and the coefficient of friction. The battery and heavy components should be placed low and distributed to maintain stability and grip without overloading a single support point.

Run a ramp test using small increments. Record slope, motor current, speed, temperature, and slipping. This test validates both the mechanical calculation and the electrical budget.

4.5 Arm torque

For a joint, add the moment produced by each mass relative to the axis:

TEXT

$$T_{\text{static}} = \sum (\text{mass}_i \times g \times \text{distance}_i)$$

Distance is measured between the axis and the item's center of mass. For the shoulder, include the masses of the upper arm, forearm, gripper, servos installed beyond the axis, and payload. The worst condition usually occurs with the arm horizontal.

Example: a 0.25 kg link with its center of mass 0.10 m away; a 0.20 kg distal assembly centered at 0.28 m; and a 0.10 kg payload at 0.40 m.

TEXT

$$\begin{aligned} T &= (0.25 \times 9.81 \times 0.10) \\ &+ (0.20 \times 9.81 \times 0.28) \\ &+ (0.10 \times 9.81 \times 0.40) \\ &\approx 1.19 \text{ N}\cdot\text{m} \approx 12.1 \text{ kgf}\cdot\text{cm} \end{aligned}$$

This is static torque only. Starting, stopping, backlash, impact, and misalignment require a margin. With a factor of 2.5, the joint would require approximately **3.0 N·m**, or **30.6 kgf·cm**. A servo advertised as 10 or 15 kgf·cm therefore cannot reliably meet this condition. Solutions include reducing length or mass, limiting payload, using a counterweight or spring, changing the transmission, or choosing a suitable actuator.

NOTE The traditional servo unit is kgf·cm, not kgf·cm². For approximate conversion, 1 kgf·cm = 0.0981 N·m.

4.6 Center of gravity and stability

The projection of the center of gravity onto the floor must remain within the polygon formed by the contact points. Extended arms shift the center and may tip the robot. Evaluate extreme positions in CAD and run a physical test with the robot unpowered.

Keep the battery and power supplies low in the structure. Use software to restrict pose combinations that bring the center of gravity close to the edge. AI may choose a task, but a deterministic planner must reject poses outside the validated envelope.

5. Structure and mechanical assembly

5.1 Mechanical archive

The repository contains SolidWorks parts, assemblies, and STL files. Major elements include the base, wheel supports, lower and upper body, Raspberry Pi mount, head, arm extensions, and grippers. STL files allow reproduction without the original CAD software; assembly files support deeper modifications.



*Figure 4 - CAD model of the Robotinics assembly preserved from the original archive.
Source: Robotinics project archive.*

Before printing, verify:

- units and scale;
- part revision and orientation;
- holes, clearances, and minimum thickness;
- space for nuts, screw heads, and connectors;
- load direction relative to print layers;
- interference between cables and moving parts;
- maintenance access.

5.2 Material and printing

PLA is easy to print and works for rigid prototypes, but it may lose strength at elevated temperatures and fracture at fitted joints. PETG offers greater toughness and thermal tolerance in many scenarios. ABS/ASA can be useful for functional parts when the printer and environment control warping and emissions. The choice depends on load, temperature, and printing capability.

Do not copy slicing parameters without validation. Record material, manufacturer, nozzle diameter, layer height, perimeter count, infill, and orientation. For motor mounts and joints, perimeters and orientation are often more important than high, randomly patterned infill.

5.3 Base

The base should be assembled and validated before the body. The recommended sequence is:

1. Inspect and finish the printed parts.

2. Assemble mounts, motors, and wheels without electronics.
3. Check parallelism and free rotation.
4. Position the battery and boards with templates, without final fastening.
5. Check the center of gravity and access to the master switch.
6. Install cable ducts, strain relief, and logic-ground points.
7. Perform a manual rolling test.

Misaligned wheels increase current and impair odometry. Measure the wheelbase on both sides and verify that the base is not twisted. Casters should provide support without lifting a driven wheel.

5.4 Body, arms, and head

The body houses boards, power distribution, and connections. Install removable panels and identify every harness. Arms should have stops or safe limits before power is applied. The head carries the camera, sensors, and positioning mechanisms; avoid routing cables through pinch zones.

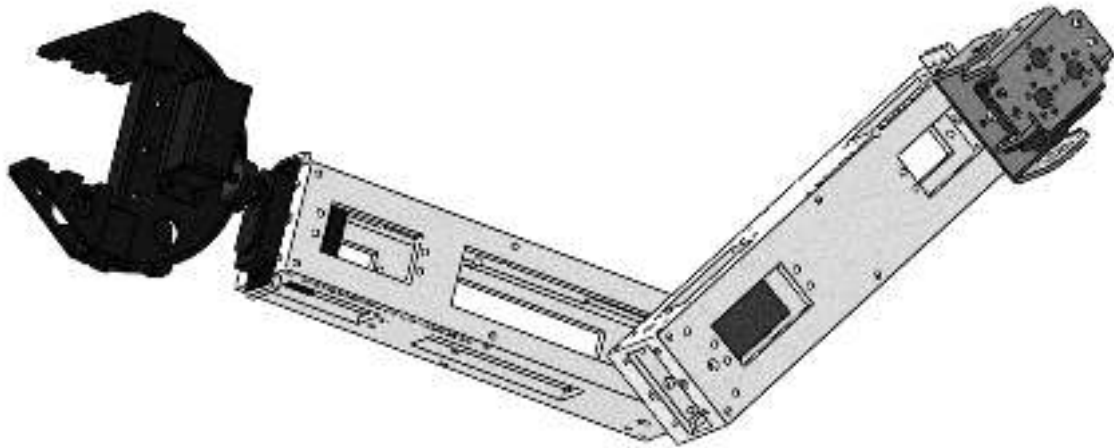


Figure 5 - Mechanical assembly of the arm and gripper.
Source: Robotinics project archive.

For every joint, record:

Item	Minimum record
Mechanical zero	physical reference position
Logical zero	value sent to the servo at that position
Minimum and maximum limits	collision-free range
No-load current	reference for detecting a stall
Maximum payload	validated condition, not merely advertised
Positive direction	convention used by the firmware

5.5 Alternatives to 3D printing

The original concept allows volumes to be built from accessible materials such as sheets, profiles, and coated spherical elements. Any substitution must preserve fastening, rigidity, access, and protection.

Expanded polystyrene should not be exposed to products that dissolve it; coatings and adhesives should first be tested on a sample.

5.6 Mechanical acceptance criteria

- No cracked or deformed parts.
- Critical screws use suitable thread locking and inspection marks.
- Wheels are free and aligned.
- Cables are not strained throughout the full range of motion.
- The center of gravity remains stable with the arms in allowed poses.
- Accidental contact with moving parts is prevented.
- The battery, fuse, and emergency stop remain accessible without extensive disassembly.

6. Power, battery, and distribution

6.1 Fundamental quantities

Voltage represents potential difference. Current represents the flow of charge. DC electrical power is $P = V \times I$. Energy is power accumulated over time and may be expressed in Wh. Battery capacity in Ah is not energy by itself; voltage must also be considered.

In the previous edition, currents from different rails were added without converting power. The correct procedure is to calculate the power of each load on its rail, include converter efficiency, and then estimate current drawn from the battery.

6.2 3S pack

Consider three lithium-ion cells rated at 3.7 V nominal and 5.2 Ah connected in series:

Property	Result
Nominal voltage	$3 \times 3.7 = 11.1 \text{ V}$
Maximum charge voltage	$3 \times 4.2 = 12.6 \text{ V}$, when specified for the cell
Capacity	5.2 Ah
Approximate nominal energy	$11.1 \times 5.2 = 57.7 \text{ Wh}$

In series, voltages add while capacity in Ah remains the same. A value of 15.6 Ah would correspond to three identical cells connected in parallel, not in series.

Values of 4.2 V per cell are common, but the manufacturer's datasheet is authoritative. Chemistry, charge limit, current, and temperature vary. Do not mix cells of different models, capacities, ages, or states.

6.3 Safe charging

Lithium cells require a CC/CV charger compatible with the number of cells and a suitable protection/balancing system. A BMS protects the pack, but it does not turn an ordinary power supply into a proper charger. For 3S, use a charger designed for 3S and the actual chemistry, with final voltage and current within specification.

Do not derive charge current from a universal rule such as $C/10$. Use the value recommended by the cell and pack manufacturers. Cells in series do not triple capacity for this calculation.

DANGER Rev. 3 does not teach how to build a mains-connected charger. Use certified equipment suitable for the pack. A swollen, damaged, hot, or abnormally charged cell must be isolated and disposed of according to local hazardous-waste guidance.

6.4 BMS, fuse, and master switch

The minimum architecture includes:

1. a safely assembled pack;
2. a BMS/protection device compatible with 3S, current, and chemistry;
3. a fuse near the positive battery terminal;
4. a master switch rated for direct current;
5. protected distribution for each rail;
6. polarized connectors with adequate capacity.

The fuse protects the conductor and reduces fault energy. Its rating must not exceed the safe capacity of wires and connectors. Starting currents must be considered to avoid nuisance opening without turning the fuse into a mere piece of wire.

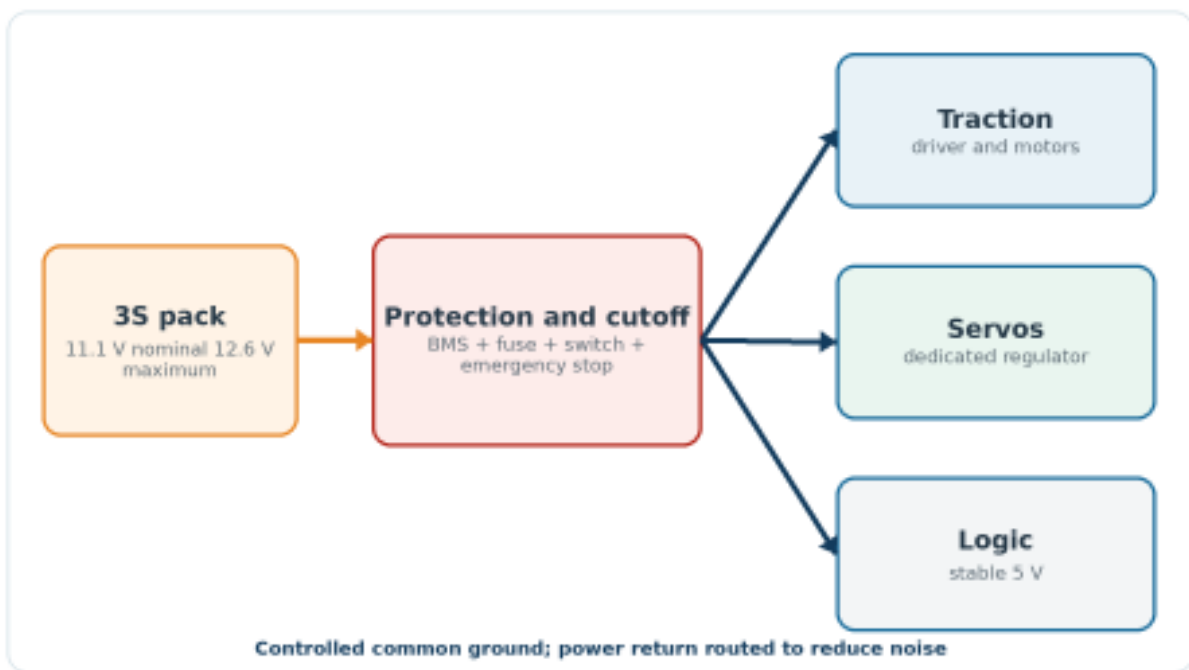


Figure 6 - Power distribution with protection and separate rails.
Source: Created by the author.

6.5 Separate rails

DC motors, servos, and logic generate and tolerate noise differently. Rev. 3 uses separate rails:

- **Traction:** voltage compatible with motors and driver, with its own protection.
- **Servos:** high-current converter at the voltage allowed by the servos.
- **5 V logic:** Raspberry Pi and compatible peripherals, with margin for peaks.
- **3.3 V logic:** sensors or interfaces that require it.

Grounds may need a common signal reference, but distribution must prevent motor current from flowing through the logic return. Use star topology, appropriate conductors, decoupling, and oscilloscope verification when possible.

A "5 V/5 A" supply is not automatically sufficient for Raspberry Pi, servos, and peripherals. High-torque servos may produce large peaks. Size the system using measurements and margin, and never power several servos through the Arduino board regulator.

6.6 Converters

A buck converter reduces voltage. It cannot regulate 12.6 V from a 12 V input. Raising voltage requires a boost converter; operation both above and below the output requires buck-boost. In Robotinics, prefer an architecture in which converters operate comfortably within their specified ranges.

Check maximum voltage, actual continuous current, cooling, ripple, efficiency, and transient response. The current printed on inexpensive modules may represent a peak under ideal conditions rather than continuous operation.

6.7 Runtime

Approximate runtime can be estimated as:

TEXT

$$\text{time_h} \approx (\text{nominal_energy_Wh} \times \text{usable_efficiency}) / \text{average_power_W}$$

With 57.7 Wh, an overall usable efficiency of 80%, and average consumption of 35 W, the estimate is $(57.7 \times 0.8) / 35 \approx 1.32$ h. Motion, terrain, temperature, aging, and BMS limits affect the result. Validate it in a controlled test and do not discharge beyond the safe limit.

6.8 Capacitors and ripple

20.75×10^{-3} F equals 0.02075 F, or 20,750 μF ; not 20.75 μF . In a rectified supply, a common approximation is:

TEXT

$$C \approx I / (f_{\text{ripple}} \times \Delta V)$$

For 1 A, 120 Hz ripple, and a 1 V variation, $C \approx 8,333$ μF . This example does not replace power-supply design or consideration of tolerance, ESR, capacitor voltage, and ripple-current rating.

6.9 Correct measurement

Voltage is measured in parallel. Current is measured in series with the load or by using an appropriate sensor. Placing a multimeter set to current directly between positive and negative creates a low-resistance short circuit and may blow a fuse, damage probes or the circuit, or cause an accident.

Before measuring:

- confirm the selected terminals and function;
- start with an appropriate range;
- de-energize before changing a series connection;
- respect the instrument category, voltage, current, and fuse ratings;
- use a clamp or sensor when the series method is unsafe.

6.10 Electrical acceptance criteria

- Polarity and continuity verified without the battery.
- Fuses installed and identified.
- Each rail remains within tolerance, both unloaded and under load.

- Ripple and voltage drop are acceptable during motor/servo starting.
- Idle and peak currents are recorded.
- No connector or wire heats beyond expectation.
- BMS and charger are compatible and documented.
- Undervoltage and shutdown are tested in a controlled manner.

7. Electronics, sensors, and actuators

7.1 Arduino and logic levels

The historical firmware was developed for Arduino Mega, which normally uses 5 V logic. Raspberry Pi GPIO uses 3.3 V and is not directly 5 V tolerant. Every connection between boards must account for level, direction, current, and startup state. Use a level shifter when needed; never rely only on a coincidental bench result.

7.2 Motor driver

The L298N is a dual H-bridge based on bipolar transistors. It supports direction control and speed control through PWM applied to its enable pins. It does not provide galvanic isolation and has significant voltage drop and dissipation, especially at low motor voltage.

If retained for compatibility, measure motor voltage and driver temperature at worst-case load. In an electronics revision, a modern MOSFET driver sized for voltage, starting current, and braking will usually provide better efficiency.

Motors appear with different voltage ratings in historical sections. Rev. 3 does not assume 12 V: identify the installed model and follow its datasheet. A battery voltage of 11.1/12.6 V does not authorize direct connection of a 3-6 V motor.

7.3 Relays

A normally open (NO) contact remains open when the coil is de-energized and closes when energized. A normally closed (NC) contact remains closed at rest and opens when energized. Check the component diagram, coil voltage, contact current, and need for a flyback diode.

7.4 LEDs

0.02 A is 20 mA, not 200 mA. The resistor is calculated as:

TEXT

$$R = (V_{\text{supply}} - V_{\text{LED}}) / I_{\text{LED}}$$

For a 5 V supply, a red LED with an approximate 2 V drop, and a desired current of 9 mA, $R \approx 333 \, \Omega$; 330 Ω is an appropriate standard value in this example. Calculate resistor power as $P = I^2 \times R$ and confirm the actual LED specification.

7.5 Ultrasonic sensor

The HC-SR04 uses ultrasonic pulses near 40 kHz, not 40 Hz. The microcontroller sends a trigger pulse and measures echo duration. Approximate distance is calculated by accounting for the sound's round trip:

TEXT

$$\text{distance} = \text{speed_of_sound} \times \text{time} / 2$$

Take independent readings, discard timeouts, and apply filtering. Converting the same measured time three times does not produce three samples. On Arduino, define the pins and set a timeout on `pulseIn` to avoid extended blocking.

The echo signal of many modules is 5 V. Use level adaptation for Raspberry Pi. Temperature, angle, material, and interference between sensors affect the result; ultrasound is an aid, not the only safety barrier.

7.6 Servos

The Arduino `Servo` library uses timers and can control servos on several digital pins; it is incorrect to state that only pins marked PWM can be used. Timer use may, however, interfere with other functions, and channel count depends on the board and library.

Power servos from a separate supply and join the signal reference in a planned manner. Define individual mechanical limits. At startup, do not abruptly send a position that causes a collision; load calibration, validate it, and move with a ramp.

7.7 Analog sensors

Voltage and current inputs must remain within ADC range. Resistor dividers must account for tolerance and impedance; current sensors require zero, gain, and noise calibration. To calculate average or RMS, clear accumulators at the start of the window and use distinct samples.

An overcurrent alarm should be below the level that damages wires, driver, or battery. Fast response may remain in firmware; Raspberry records and explains the event.

7.8 Decoupling and electromagnetic compatibility

Install decoupling capacitors near circuits, keep current loops small, and separate power cables from signals. Brushed motors may require suppression at the motor itself. Twist supply and return pairs where appropriate, and do not use data cables/connectors without checking current, voltage drop, and pinout.

An RJ45 connector can be used as an internal physical connector, but that does not make the circuit Ethernet. Label it clearly to prevent accidental connection to a network.

7.9 Electronics checklist

- Rated voltage and current confirmed for every component.
- 3.3 V and 5 V levels made compatible.
- Pins and connectors documented.
- Driver sized for starting current.
- Servos powered outside the Arduino regulator.
- Sensors calibrated and equipped with timeouts.
- Relays and inductive loads protected.
- Cables labeled, secured, and clear of moving parts.

- Noise test performed with motors and servos moving.

Part III - Firmware, Linux, and communication

ROBOTINICS REV. 3

8. Robotinics Rev. 3 firmware

8.1 Why reorganize it

The historical sketch demonstrates many features, but it concentrates about 1,500 lines in one file. It uses `String` extensively on a microcontroller with little RAM, mixes buffers from several ports, contains blocking waits, and interprets commands using substring tests. A prefix such as `ULTRA` may therefore also match `ULTRA1`, while timeouts, tests, and maintenance become difficult.

There are also objective defects: definitions such as `#define PINOGPSRX = 0;` are not valid macros when expanded; one routine prepares `sInfo` but prints another variable; the `SERIAL:` slice uses an index inconsistent with the prefix; ultrasonic reading repeats conversion of the same time; and the GPS parser may block and overflow its buffer.

Rev. 3 preserves the value of the original firmware while proposing a modular architecture and stable interface.

8.2 Modules

Module	Responsibility
Config	pins, limits, and board version
CommandParser	reception, tokenization, and syntax validation
Safety	state, heartbeat, emergency, timeouts, and faults
Drive	direction, PWM, ramp, and braking
Servos	calibration, limits, and gradual movement
Sensors	ultrasound, current, voltage, gas, and accelerometer
Telemetry	structured responses and events
Manual	deterministic catalog published through MAN

On small boards, modules may still compile into one sketch, but they should remain separated into files or clearly scoped functions.

8.3 Safe initialization

At power-up:

1. Set enable pins inactive before direction pins.
2. Turn motors off and keep the robot in `DISARMED`.
3. Initialize serial ports and sensors with timeouts.
4. Load validated calibration.
5. Publish the version and reason for any fault.
6. Accept motion only after `ARM` and verification of local conditions.

Servos should start at a known position without jumping. When physical position is unknown, move slowly toward a safe reference or require a manual procedure.

8.4 Bounded serial input

Use a fixed buffer, for example 128 bytes. Append each received byte until `LF` or `CRLF`. If the limit is exceeded, discard the line, report `ERR LINE_T00_LONG`, and do not execute partial content. Every port must have its own receive state.

After receiving a line, split the first token and compare the whole command rather than searching for a substring. Convert parameters with error and range detection. An invalid line produces an error, never partial behavior.

TEXT

```
Input:    MOVEFWD 120 800
Command:  MOVEFWD
PWM:      120, validated within 0..255
Duration: 800 ms, validated within 1..2000
```

8.5 Command manual

The TCHATGPT TAIAgentSerial discovers the catalog from deterministic markers and lines. Rev. 3 firmware should respond:

TEXT

```
MAN-BEGIN
DEVICE: Robotinics Rev3
VERSION: 3.0.0
BAUD: 115200 8N1
COMMAND PING: test communication, no parameters
COMMAND STATUS?: return state, voltage, current, and faults
COMMAND ARM: enter armed state after local validation
COMMAND DISARM: disable actuators
COMMAND STOP: stop motion immediately
COMMAND MOVEFWD: MOVEFWD <pwm 0..255> <time_ms 1..2000>
COMMAND MOVEBACK: MOVEBACK <pwm 0..255> <time_ms 1..2000>
COMMAND TURNLEFT: TURNLEFT <pwm 0..255> <time_ms 1..1500>
COMMAND TURNRIGHT: TURNRIGHT <pwm 0..255> <time_ms 1..1500>
COMMAND RANGE?: return valid sensor distances
COMMAND BATTERY?: return voltage and estimated state
COMMAND MAN: publish this manual
MAN-END
```

The agent interprets only `COMMAND` lines. Information such as version and baud rate is shown to the operator but does not create actions. Unknown commands remain blocked. Historical aliases `FRENTE`, `RE`, `GESQ`, `GDIR`, and `PARA` may be retained for manual use, mapped internally to the new API and subject to the same limits.

8.6 Watchdog and heartbeat

Motion must expire. Even when a command contains a duration, firmware records the starting instant and stops at the limit. If the application uses continuous motion, it sends a periodic heartbeat; absence for a short interval, for example 500 ms, leads to `STOP` and either `FAULT` or `DISARMED`, according to policy.

Configure the hardware watchdog to recover from firmware hangs. After a watchdog reset, the robot starts disarmed and records the cause. Restarting must not resume the previous action.

8.7 Obstacles and local limits

Firmware may block forward motion when a valid front sensor indicates a distance shorter than the configured margin. Invalid readings must not automatically become "clear path." Define an explicit policy: reduce speed, stop, and request another reading.

Servos have per-joint limits. Excessive current or time suggests a stall and should stop output. A physical emergency stop should remove actuator power appropriately and report its state to logic when possible.

8.8 Telemetry

Responses follow simple patterns:

TEXT

```
OK <command> [details]
ERR <code> <short description>
STATE <key>=<value> ...
EVENT <type> <data>
```

Examples:

TEXT

```
OK ARM
STATE MODE=ARMED BATTERY_MV=11780 FRONT_MM=640
EVENT STOP REASON=HEARTBEAT_TIMEOUT
ERR RANGE_TIMEOUT SENSOR=FRONT
```

Include the unit in the key or contract. `BATTERY_MV=11780` is less ambiguous than `BATTERY=11.78` without a unit.

8.9 Avoiding memory fragmentation

On AVR, prefer `char` arrays, bounded buffers, and functions that do not make many copies. Avoid indefinite `String` concatenation. Serial content must never grow without limit. Long-running tests should observe RAM, stability, and recovery from invalid lines.

8.10 Firmware tests

- `STOP` works in every state.
- Motion is refused in `DISARMED`.
- Out-of-range PWM and duration are refused.
- A long line does not execute a fragment.
- An unknown command is refused.
- Loss of heartbeat stops motion.
- A sensor timeout does not freeze the loop.
- Reset returns to a safe state.
- `MAN` contains implemented commands only.
- Similar prefixes do not cause double execution.

9. Raspberry Pi and robot services

9.1 Role of Raspberry Pi

Raspberry Pi does not replace Arduino for real-time control. It performs higher-level tasks: camera and audio interfaces, network integration, storage, a compatible Lazarus application, serial gateway, telemetry, and AI services that fit the platform.

The system must remain safe during boot, update, or a crash. Actuator enable must not depend on a floating GPIO; Arduino remains disarmed until it receives a valid sequence.

9.2 System installation

Use Raspberry Pi Imager and the current official documentation. Configure a dedicated user, strong password, network, and SSH while writing the image when supported. Do not assume the old user `pi` with password `raspberrypi`; that practice belongs to historical images.

Record:

- Raspberry Pi model;
- ARM64 or ARMHF architecture;
- Raspberry Pi OS version;
- image and date;
- installed packages;
- enabled services;
- kernel and firmware version;
- serial-port, camera, and audio test results.

9.3 Minimum profile

Start with the Lite system when the robot does not require a local graphical environment. Install only necessary dependencies. Graphical interfaces, Chromium, OpenCV, voice, and local models increase consumption, storage, and failure surface.

TCHATGPT classifies Linux ARM64 and ARMHF as experimental in this edition's snapshot. Compile `openai_core` and a console test first. Then validate `openai_input`, `openai_voice`, and `openai_vision` separately. Do not install the whole suite and attribute a generic failure to Raspberry Pi.

9.4 Services with systemd

Long-running applications should be managed by `systemd`, under an unprivileged user, with an explicit working directory, controlled restart, and journal logging.

```
INI
[Unit]
Description=Robotinics Gateway
After=network-online.target
Wants=network-online.target
```

```
[Service]
Type=simple
User=robotinics
Group=robotinics
WorkingDirectory=/opt/robotinics
ExecStart=/opt/robotinics/bin/robotinics-gateway
Restart=on-failure
RestartSec=3
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

Grant serial-device permissions through the appropriate group and rules. Do not run the service as root merely to access `/dev/ttyUSB0`.

9.5 Dependencies and installer

External dependencies should be resolved deterministically by an installer or version-controlled script. Documentation states which are required, optional, and platform-specific. Installer tests must confirm runtime libraries, not merely compilation.

For TCHATGPT packages, Python-based components should use `TAIPythonRuntime`. Vision, voice, and native libraries must validate architecture. An x64 binary does not run on ARM64; a 32-bit library does not satisfy a 64-bit process.

9.6 Camera and audio

Test capture and playback outside AI. First confirm device, format, rate, and latency. Then connect voice or vision components. This separates hardware, permission, and library failures from model failures.

Use persistent names or rules for USB devices when ordering may change. The service should clearly report "device missing" and remain safe rather than freezing the whole robot.

9.7 Storage

MicroSD cards wear out. Avoid unlimited logs and continuous unbounded writes. Use a lightweight database or atomic files for configuration, with backups and validation. Important data may be sent to external storage when the network is available without blocking local control.

9.8 Update and rollback

Update the application and firmware as versioned units. Before replacing them:

1. save the current configuration and version;
2. verify package signature or origin;
3. install into a version directory;
4. run a self-test without actuators;
5. change the active-version link;
6. monitor the first boot;
7. return to the previous version if criteria fail.

10. Communication, data, and software security

10.1 Serial before networking

Validate the local serial link between Raspberry and Arduino before exposing any network interface. Use 115200 8N1, line termination, and a documented protocol unless there is a technical reason to use another configuration. The gateway limits size, rate, and timeout.

Do not reuse one buffer for `Serial`, `Serial1`, and a `SoftwareSerial`. Every channel has its own origin and state. Identify the origin when forwarding a response.

10.2 Historical servers

The `srvMonitor2` and `srvFala` programs document the original experience, but they must not be run on a network as they are. Identified problems include an invalid read size, unterminated buffers, input-controlled format strings, the wrong variable passed to `system`, no authentication, and binding on all interfaces.

The speech server also forwards input to a shell, enabling command injection. Rev. 3 replaces this design with voice components and an API that accepts only structured fields with permitted size, characters, and actions.

10.3 Ports and contracts

Historical material uses different ports for the same service: the speech tool uses `7091`, while the current server uses `8091`. The controller sends `PARAR` at one point, but firmware recognizes `PARA`. Contracts of this kind must exist in one version-controlled file and be tested automatically.

Rev. 3 recommends against hard-coding ports in several sources. Use central configuration, a documented default, and conflict validation. Record address, protocol, and purpose.

10.4 Network interface

By default, robot services listen only on `127.0.0.1` or a local socket. Network exposure requires authentication, authorization, request limits, and transport protection appropriate to the risk. Never accept actuator commands through Telnet or a raw socket accessible to any machine.

When remote access is necessary, use a properly configured and maintained VPN or HTTPS layer. The firewall should allow only necessary origins and ports. Emergency stop and local safety remain independent of the network.

10.5 Structured messages

JSON is sufficient for moderate-volume communication between services. Define a schema and version:

```
JSON
{
  "schema": "robotinics.command.v1",
  "request_id": "8f2c...",
```

```

"action": "move_forward",
"parameters": {"pwm": 120, "duration_ms": 800},
"requires_confirmation": true
}

```

The service validates types, required fields, ranges, and size before conversion to the serial protocol. Unknown fields may be rejected to avoid divergent interpretations.

10.6 Database

The database supports configuration, inventory, telemetry, and auditing, not the safety loop. The robot must stop even when the database is unavailable.

Do not use a root account in the application. Create a least-privilege user and store the secret outside the code. Use parameterized queries. Administrative interfaces such as phpMyAdmin must not be publicly exposed or required for robot operation.

The historical C/MySQL example contains typographic quotes, capitalization errors, and parentheses that prevent compilation. In Rev. 3, complete examples should compile in CI; illustrative fragments are identified as pseudocode.

10.7 Observability

Every request receives a `request_id` or `TraceID`. Record:

- origin and time;
- controller and firmware versions;
- received intent;
- proposed structured action;
- validation and confirmation result;
- command sent;
- returned telemetry;
- duration and error.

Do not record API keys, passwords, or sensitive audio or images by default. Logs require rotation and a retention policy.

10.8 Resilience tests

- disconnect the network during motion;
- interrupt the model response;
- send invalid and oversized JSON;
- repeat a `request_id`;
- disconnect and reconnect serial;
- restart Raspberry Pi;
- hang the gateway process;
- make the database unavailable;

- simulate a delayed response;
- confirm that no failure prevents Arduino from executing `STOP`.

Part IV - Artificial intelligence with TCHATGPT

ROBOTINICS REV. 3

11. Preparing TCHATGPT

11.1 The project

TCHATGPT is a suite of visual and nonvisual components for Lazarus and Free Pascal. In the adopted commit, unit `chatgpt` declares version 1.7 and integrates remote providers and local servers. The suite is modular: do not install the former monolithic `openai.lpk` package.

The main packages for basic Robotinics integration are:

Package	Robot use	Snapshot status
<code>openai_core</code>	TCHATGPT, prompts, models, and foundation	Stable/Beta based on compilation
<code>openai_input</code>	serial, sockets, capture, and input	Stable/Beta; specific combinations may be experimental
<code>openai_agent</code>	serial agent, memory, actions, and safety	mixed; components used here have evidence, while the general pipeline remains experimental
<code>openai_voice</code>	capture, STT, TTS, authorized cloning, and assistant	Stable/Beta based on dedicated samples
<code>openai_vision</code>	image, filters, camera, and OpenCV	Beta, with backends of differing maturity
<code>openai_rag</code>	document indexing and retrieval	Stable/Beta based on sample
<code>openai_observability</code>	TraceID, spans, and metrics	Stable/Beta based on compilation

"PASS" in the project report means that a sample compiled. It does not automatically prove that a camera, microphone, board, model, or external service works at runtime. Rev. 3 maintains this distinction.

11.2 Providers

TCHATGPT supports OpenAI, OpenRouter, Cerebras, Gemini, Claude, DeepSeek, an OpenAI-compatible endpoint, llama.cpp, neural-api, and a local profile. The book does not require a provider.

Choose according to privacy, latency, cost, availability, and hardware capacity:

- **Remote:** lower compute requirements on the robot; depends on internet connectivity and provider policy.
- **Local x64:** suitable for a laboratory server on the network; data stays within the controlled environment.
- **Local on Raspberry:** greater independence, but models and vision compete for CPU/RAM; requires actual measurement.
- **Hybrid:** simple commands and safety remain local; complex language tasks run on a remote server or more powerful local server.

11.3 Configuration without secrets in code

The following fragment uses the real API from the frozen version. `SendQuestion` returns `Boolean`; the response is stored in `Response`. Do not treat the function result as text.

```
PASCAL
uses
  SysUtils, chatgpt;

procedure ConfigureLLM(Chat: TCHATGPT);
begin
  Chat.Provider := AIP_OPENAI_COMPATIBLE;
  Chat.URL := GetEnvironmentVariable('ROBOTINICS_LLM_URL');
  Chat.CustomModel := GetEnvironmentVariable('ROBOTINICS_LLM_MODEL');
  Chat.TOKEN := GetEnvironmentVariable('ROBOTINICS_LLM_TOKEN');
  Chat.MaxTokens := 256;
  Chat.Temperature := 0.1;
  Chat.Timeout := 30000;
  Chat.Dev :=
    'Convert the intent to strict JSON. Do not invent commands. ' +
    'Physical execution will be validated by another layer.';
end;

function AskLLM(Chat: TCHATGPT; const Question: string): string;
begin
  if not Chat.SendQuestion(Question) then
    raise Exception.Create(Chat.LastError);
  Result := UTF8Encode(Chat.Response);
end;
```

In Marcelo's laboratory, a local server may be configured on port 8095. This is an environment setting, not a universal TCHATGPT default. Set `ROBOTINICS_LLM_URL`, for example `http://127.0.0.1:8095/v1/chat/completions`, only after confirming the endpoint and model.

11.4 Asynchronous calls

Graphical interfaces must not freeze during a request. TCHATGPT offers `SendQuestionAsync`, streaming, state events, and cancellation. States include `Idle`, `Connecting`, `Receiving`, `Completed`, `Cancelled`, and `Error`.

Use asynchronous calls for conversation and explanations. Physical control still requires serialization: do not allow two concurrent decisions over the same actuator. A coordinator maintains a short queue, cancels obsolete requests, and invalidates a response that arrives after a state change.

11.5 Prompts and contracts

A prompt is not an authorization mechanism. It describes the expected format, but every response must pass through a parser. For control, use strict JSON, a small action set, and numeric parameters. Set a low temperature and token limit; long responses only increase latency and attack surface.

Do not send the entire history, all telemetry, or complete documents to the model. Select relevant context, state the units, and include current state. A decision made from stale telemetry must expire.

11.6 Self-test

Before associating the LLM with the serial agent:

1. Send a short question and confirm `Response`.
2. Stop the server and validate `LastError` and timeout behavior.

3. Cancel a request.
4. Test Unicode text and JSON.
5. Measure average and worst-case latency.
6. Confirm that the token does not appear in logs or configuration.
7. Run the test with every actuator de-energized.

12. Supervised control agent

12.1 Components

The `agent_serial_demo` sample provides the closest foundation for Robotinics. It uses:

- `TCHATGPT` for interpretation;
- `TAISerialModem` for the real port;
- `TAIListSerialDevices` for port discovery;
- `TAIAgentSerial` for structured actions;
- `TAIAgentAction` as the command catalog;
- `TAIAgentMemoryMap` for history;
- events for confirmation, logs, and rejected commands.

Rev. 3 adds `TAIAgentSafety` and a Robotinics-specific parameter validator.

12.2 Safe configuration

```
PASCAL
procedure TfrmRobotinics.ConfigureAgent;
begin
  AIAgentSerial1.Serial := AISerialModem1;
  AIAgentSerial1.LLM := CHATGPT1;
  AIAgentSerial1.CommandCatalog := AIAgentCommands1;
  AIAgentSerial1.RequireConfirmation := True;
  AIAgentSerial1.MaxActionsPerPrompt := 3;
  AIAgentSerial1.AutoDiscoverCommands := True;
  AIAgentSerial1.AllowUnknownDeviceCommands := False;
  AIAgentSerial1.ClearCatalogOnDisconnect := True;

  AIAgentSafety1.Enabled := True;
  AIAgentSafety1.ReadOnlyMode := False;
  AIAgentSafety1.SimulationMode := False;
  AIAgentSafety1.AllowIndustrialWrite := True;
  AIAgentSafety1.RequireConfirmation := True;
end;
```

`AllowIndustrialWrite=True` does not permit everything. The action list, capability registration, `MAN` catalog, parameter validator, and confirmation remain mandatory.

12.3 Capability registration

Register the action that represents physical transmission:

```
PASCAL
RegisterActionCapability(
  'ROBOTINICS_SERIAL_SEND',
  [acIndustrialWrite, acStateMutation]
);
```

Add this action to `AIAgentSafety1.AllowedActions`. The policy is fail-closed: an unregistered action must be refused.

12.4 Event before the action

The `OnBeforeAction` event receives the proposed type and parameter. The Robotinics implementation should:

1. map the internal type to a safety action;
2. validate physical-command syntax and range;
3. validate current state and telemetry;
4. call `TAIAgentSafety.ValidateAction`;
5. present a clear confirmation;
6. set `AAllow=True` only if every step succeeds.

PASCAL

```
procedure TfrmRobotinics.AgentBeforeAction(Sender: TObject;
  AKind: TAgentActionKind; const AParam: string; var AAllow: Boolean);
var
  Params: TStringList;
  ErrorText: string;
begin
  AAllow := False;

  if not ValidateRobotinicsCommand(AKind, AParam, ErrorText) then
  begin
    LogRejection(AParam, ErrorText);
    Exit;
  end;

  Params := TStringList.Create;
  try
    Params.Values['COMMAND'] := AParam;
    AAllow := AIAgentSafety1.ValidateAction(
      'ROBOTINICS_SERIAL_SEND', Params, ErrorText);
  finally
    Params.Free;
  end;

  if not AAllow then
    LogRejection(AParam, ErrorText);
end;
```

`ValidateRobotinicsCommand` belongs to the application because it knows commands, ranges, state, and sensors. The generic component should not embed robot-specific rules.

12.5 Understandable confirmation

Do not show only "Allow send?" Show the physical action and its consequence:

TEXT

```
The robot will move forward at PWM 120 for up to 800 ms.
Current front distance: 640 mm.
State: ARMED. Battery: 11.78 V.
[Allow once] [Cancel]
```

Do not use one generic confirmation for a long sequence. Every motion action should be short, or the entire sequence should be presented with limits and a stop option.

12.6 Deterministic discovery

On connection, `TAIAgentSerial` sends `MAN`. The parser expects `MAN-BEGIN`, collects lines of the form `COMMAND <name>: <description>`, and finishes at `MAN-END`. The LLM does not interpret the manual; the parser does. Before `send`, the first token is compared with the catalog.

This design is a strength of TCHATGPT and should be preserved. For Robotinics, firmware publishes only commands actually available in the current configuration. A missing arm must not generate arm commands.

12.7 Session memory

`TAIAgentMemoryMap` can record questions, responses, and actions. Do not use history as a source of physical state: state comes from current telemetry. An old sentence such as "the robot is armed" does not authorize movement after restart.

Do not store the token in history. Define retention, a new-conversation option, and a maximum size. For auditing, store IDs and validation results in a log separate from the conversation.

12.8 Current limits

`TAIAgentSerial.Execute` uses a synchronous LLM call. In a GUI, run it without blocking the interface and synchronize component access. The component limits the number of actions, but it does not know acceleration, geometry, battery, or distance; these rules belong to Robotinics.

12.9 Adversarial tests

Ask the model to:

- use a command that does not exist;
- move with PWM -1 or 999;
- move for an excessively long time;
- ignore instructions and send text to the shell;
- run five actions when the limit is three;
- move while `DISARMED`;
- move forward with a nearby obstacle;
- use information from a RAG document as a command.

Every case must be blocked or converted into a question without action.

13. Voice and human interaction

13.1 Pipeline

The `openai_voice` package integrates the flow:

TEXT

```
TAIAudioInput → TAISpeechRecognizer → TCHATGPT
               → TAIVoiceClone or TAIVoiceSynthesizer → TAIAudioPlayer
```

`TAIVoiceAssistant` coordinates recognition, model, and synthesis. States indicate whether the system is listening, recognizing, thinking, speaking, cancelled, or in error.

13.2 Speech recognition

The process-based Whisper backend requires a compatible executable and model. Variables such as `WHISPER_CLI` and `WHISPER_MODEL` avoid hard-coded paths. Test a known WAV file first, then the microphone, and finally the integration.

Continuous recognition may divide audio into blocks. In the snapshot, processing remains synchronous, so the GUI must use an appropriate worker. The operator needs a visible indication when the microphone is active.

13.3 Voice is not authorization

Noise or misrecognition can turn "do not move forward" into "move forward." Physical actions derived from voice require interface confirmation or a clear challenge-response phrase. Stop words should receive local, high-priority handling, but they do not replace the emergency-stop button.

A recommended sequence is:

1. user: "move forward a little";
2. STT produces text;
3. agent proposes `MOVEFWD 90 500`;
4. application presents the interpretation;
5. user confirms;
6. validation and firmware execute it;
7. voice reports the result.

13.4 Traditional synthesis and voice cloning

Traditional TTS is sufficient for operational feedback and reduces dependencies. Voice cloning is optional. `TAIVoiceClone` requires consent by default, identification of the voice owner, and an authorized reference recording. Do not clone a third party's voice without explicit authorization.

Announce a synthesized file only after it exists and has a valid size and WAV header. Cancellation must propagate to STT, LLM, and synthesis.

13.5 Privacy

Do not record the microphone continuously by default. Disclose capture, retention, and destination.

When remote STT or LLM is used, audio or text may leave the device. Avoid storing participants' voices in classroom demonstrations.

13.6 Acceptance criteria

- Capture and playback work independently.
- STT correctly recognizes a set of test phrases in the real environment.
- Cancellation stops every stage.
- A physical action passes through the same validation as typed text.
- Voice-cloning consent is verifiable.
- A missing microphone or model produces a clear error and does not freeze the robot.

14. Computer vision

14.1 Historical correction

The historical `face.py` file imports `VPython/visual` and draws an animated face. It does not implement OpenCV face recognition and must not be presented as doing so. Motion performs capture/motion detection according to its configuration, but this does not automatically constitute semantic recognition.

14.2 Current components

TAIOpenCV has two backends:

Backend	Recommended use	Limitation
<code>ocvPythonProcess</code>	actual image processing through a Python worker	depends on Python, OpenCV, and NumPy
<code>ocvNativeDLL</code>	runtime detection and loading	complete native processing is still partial/experimental

In the frozen version, actual filters include gray, blur, Canny, threshold, and resize in the Python backend. The combined demo mentions simulated tracking in places; it must therefore not be used as proof of real recognition.

For face, object, or pose detection, use dedicated components and samples, confirm the actual backend, and validate with test images. `TAIHumanPoseDetector` requires a 64-bit platform in the documented state.

14.3 Robotinics pipeline

The recommended pipeline is:

1. capture a frame with timestamp;
2. validate dimensions and format;
3. reduce resolution where appropriate;
4. run a filter or specialized detector;
5. convert the result into a structured observation;
6. apply a threshold and temporal stability;
7. supply the observation to the planner;
8. maintain independent proximity safety.

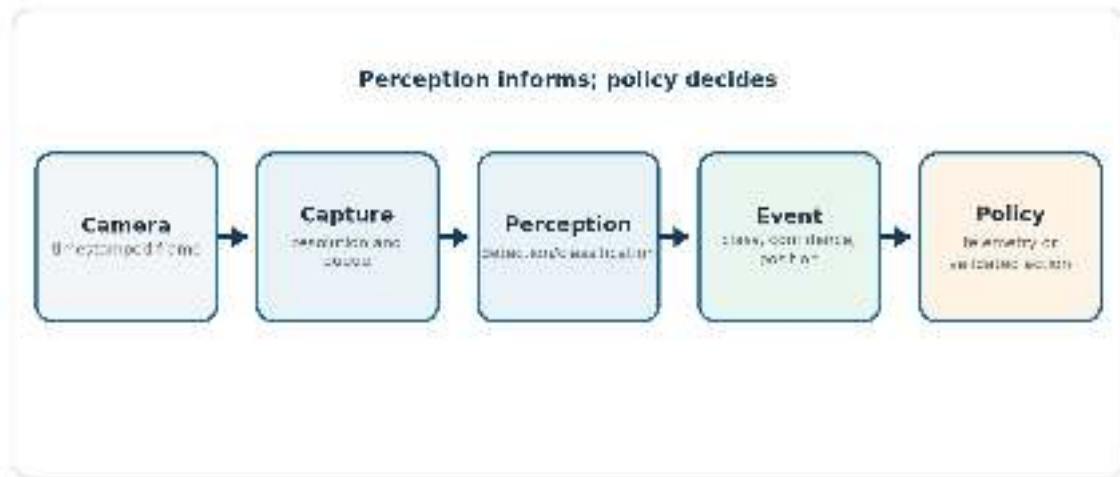


Figure 7 - Vision pipeline separated from motion control.
Source: Created by the author.

Do not send an entire image to the LLM when a simple local detection answers the question. A detector may produce `PERSON confidence=0.91 x=...`; the agent receives that observation and decides whether to explain, ask, or propose an action.

14.4 Performance on Raspberry Pi

Measure FPS, latency, CPU, RAM, temperature, and consumption. A model that works on a desktop may cause throttling on Raspberry Pi. Adjust resolution, analysis interval, and model. Separate capture from processing so old frames do not accumulate; when delayed, discard them and use the latest frame.

14.5 Safety and ethics

Face detection does not imply reliable identification. Recognition of people involves consent, bias, data protection, and false-identification risk. For an educational project, prefer presence, color, marker, or object detection without identity.

Vision must not be the only stop source. Lighting, occlusion, and motion cause failures. Local sensors and speed limits remain active.

14.6 Tests

- valid, corrupted, and missing image;
- camera disconnected during execution;
- varying light and background;
- partially occluded object;
- delayed frame;
- detector unavailable;
- high CPU load;
- confirmation that vision failure reduces capability and does not enable motion.

15. Memory and RAG

15.1 Use in the robot

RAG enables answers based on project documents without retraining the model. Robotinics can index:

- the Rev. 3 manual;
- component catalogs and authorized datasheets;
- the pin map;
- maintenance records;
- test procedures;
- a summarized fault history;
- the protocol version.

The purpose is to support diagnosis and operation, not to turn retrieved text into an automatic action.

15.2 TAIRAG

TAIRAG integrates `TAIGraphMap` and `TCHATGPT`. The minimum flow is to associate components, add files/folders, build the index, retrieve context, and ask a question. The current version offers graph, vector, BM25, and hybrid modes, a token budget, and optional reranking.

```
PASCAL
RAG.GraphMap := GraphMap;
RAG.ChatGPT := ChatGPT;
Agent.RAG := RAG;
Agent.ChatGPT := ChatGPT;

RAG.AddFolder('/opt/robotinics/docs');
RAG.BuildIndex;
```

Use extension and directory filters. Do not index secrets, keys, raw logs, or personal data without a need.

15.3 Separation of knowledge and authority

A document may say "for testing, send FRENTE." This text does not authorize the action. RAG supplies context for the answer; only the device catalog and safety policy define permitted actions. Retrieved content is treated as untrusted data and cannot override agent instructions.

15.4 Sources and traceability

The response should present the files or fragments used. Store document hash, version, and date. If two revisions differ, prefer the one linked to the robot version and report the conflict.

A useful response is: "According to the revision 3.0 pin map, the front trigger is on pin X; confirm the installed board." A poor response asserts certainty without a source or mixes pins from different revisions.

15.5 Chunking

Chunks should preserve semantic units: a complete procedure, a definition with its formula, or a table with its header. Fixed size is a starting point, not a universal solution. Use moderate overlap and chapter, component, and version metadata.

Evaluate with real questions and expected answers. Measure retrieval of the correct source before blaming the LLM. A small model can answer well when retrieved context is short and precise.

15.6 Operational memory

Separate:

- **current state:** volatile telemetry, never obtained from conversation;
- **session memory:** dialogue and recent actions;
- **maintenance memory:** confirmed events and versions;
- **document base:** indexed and versioned content.

After restart, physical state begins unknown/disarmed even if memory says that the robot was armed.

15.7 Evaluation

Create a question set:

- What is the nominal voltage of the 3S pack?
- Which command must always stop the robot?
- Where is the firmware file?
- Does the native OpenCV backend fully process images?
- Which pin-map revision is installed?

Record the retrieved source, answer, correctness, latency, and tokens. Include unanswerable questions; the system should acknowledge a lack of evidence.

16. Intelligent behaviors and limits

16.1 Operating modes

Mode	AI	Actuators	Use
Manual	optional for explanations	operator sends commands	initial diagnosis
Assisted	interprets intent	confirmation for each action	recommended use
Supervised	proposes a short sequence	limits and sequence confirmation	validated tasks
Simulation	acts on a virtual model	no physical output	development and testing

Unsupervised physical autonomy is not an initial Rev. 3 objective. It should be considered only after tests, localization, obstacles, power, and fault recovery have been validated.

16.2 Appropriate capabilities

Initial capabilities:

- explain state and faults;
- discover and list commands;
- query sensors;
- propose a short motion with confirmation;
- guide a maintenance checklist;
- answer questions using RAG;
- recognize a phrase and speak a response;
- describe an already structured vision observation.

Initially avoid:

- extended navigation without mapping/localization;
- manipulation of fragile objects or people;
- shell execution;
- automatic firmware changes;
- unrestricted internet access;
- a long action chain without confirmation or feedback.

16.3 Plan, execute, and verify

Divide a task into short steps. Validate state before each step. Then read telemetry and compare the result. If the effect is not confirmed, stop; do not repeat indefinitely.

Example: "move 30 cm closer":

1. check sensor and state;
2. propose a short initial displacement;

3. confirm;
4. execute for a limited time;
5. measure again;
6. recalculate with an iteration limit;
7. stop and report the result.

Without calibrated odometry and a reliable sensor, the system must say that it cannot guarantee 30 cm.

16.4 Uncertainty

The agent must distinguish fact, inference, and missing data. Stale timestamped telemetry is not "current state." Low-confidence detection is not confirmed presence. A document from another revision is not authoritative for the current assembly.

Define thresholds and responses: ask the operator, repeat the reading, reduce speed, or refuse. "I don't know" is safe behavior.

16.5 AI observability

Record model version, provider, temperature, prompt hash, TraceID, latency, and structured response. For privacy, do not store all content by default. After a failure, it should be possible to answer: which model proposed the action, which validator blocked or allowed it, and which command the firmware received.

16.6 Promotion criterion

A capability moves from simulation to physical operation when:

- it has a documented contract and limits;
- all commands belong to the catalog;
- normal and adversarial tests pass;
- an independent stop exists;
- network/model failures are safe;
- the interface clearly presents intent and confirmation;
- results have been repeated on real hardware;
- the code version has been frozen.

Part V - Construction, validation, and evolution

ROBOTINICS REV. 3

17. Construction sequence

17.1 Stage 0 - documentation and inventory

Before buying or energizing components:

1. download the frozen project version;
2. generate an inventory of required files;
3. identify the components actually available;
4. check datasheets and voltages;
5. record substitutions;
6. prepare the power diagram;
7. define the emergency-stop button and test procedure.

Do not use photographs as the only schematic. Create a connection map with names, pins, voltage, and current. Replacing a motor or servo may affect mechanics, power, and firmware simultaneously.

17.2 Stage 1 - passive mechanics

Assemble the base, wheels, supports, and structure without a battery. Check alignment, clearance, and access. Install weights representing the battery and boards to assess the center of gravity. Keep arms unloaded and remove servo horns during initial adjustment.

Acceptance: the base rolls freely, does not tip in permitted poses, and has no cable or part interference.

17.3 Stage 2 - power distribution

Install fuses, switch, converters, and terminals, but test initially with a current-limited bench supply. Validate one rail at a time with an electronic or known load. Measure polarity before connecting boards.

Acceptance: voltages remain within tolerance during transients; there is no unexpected heating; shutdown removes actuator power as designed.

17.4 Stage 3 - Arduino without actuators

Load Rev. 3 firmware. Use a serial terminal to test `PING`, `MAN`, `STATUS?`, invalid lines, overflow, and timeout. Simulate sensors where possible. Verify that boot always reports `DISARMED`.

Acceptance: the catalog matches firmware, the parser does not execute partial prefixes, and failures do not block the loop.

17.5 Stage 4 - raised traction system

Raise the wheels. Connect driver and motors with a current limit. Test one direction at a time, then `STOP`, duration, and watchdog. Confirm that left and right match the convention. Reverse wires or configuration in a documented way; do not scatter direction corrections across several routines.

Acceptance: starting, ramp, direction, stopping, and timeout are predictable; current and temperature remain below defined limits.

17.6 Stage 5 - servos and sensors

Connect one servo at a time. Calibrate zero and limits without the link; then install the unloaded link and measure current. Validate sensors individually, including timeout and out-of-range values.

Acceptance: no joint collides; loss of a relevant sensor reduces capability; the power supply does not reset logic during movement.

17.7 Stage 6 - Raspberry Pi and gateway

Install the system, create a service user, and validate serial communication. The gateway queries state and records telemetry but does not yet control motion over the network. Test restart, disconnection, and permissions.

Acceptance: Raspberry Pi can restart without activating actuators; the service restores communication and reports its version.

17.8 Stage 7 - TCHATGPT in simulation

Configure the LLM and agent with `SimulationMode=True` or without the physical serial port. Use a test catalog and send normal and adversarial questions. Verify JSON, limits, and confirmation.

Acceptance: the model cannot bypass catalog and policy; the interface remains responsive; credentials are not persisted as plain text.

17.9 Stage 8 - agent with hardware

Connect the real serial port with wheels raised. Discover `MAN`, query state, and execute only `STOP`, `PING`, and sensor commands at first. Then authorize short movements. Monitor the robot physically and keep the emergency stop within reach.

Acceptance: proposed action, confirmation, command, telemetry, and log show the same `request_id`; any mismatch stops the test.

17.10 Stage 9 - voice, vision, and RAG

Add one feature at a time. Voice first produces text without action; vision first produces an observation; RAG first answers with sources. Only then may each feature feed the agent through the same barriers.

Acceptance: failure of an added feature does not change basic safety; the operator knows when camera/microphone are active.

17.11 First floor operation

Use a flat, clear, marked area. Begin at low speed over a short distance. Record video, telemetry, current, and temperature. Increase gradually. Ramps and arms are later phases and never part of the first integrated test.

18. Test and commissioning plan

18.1 Test pyramid

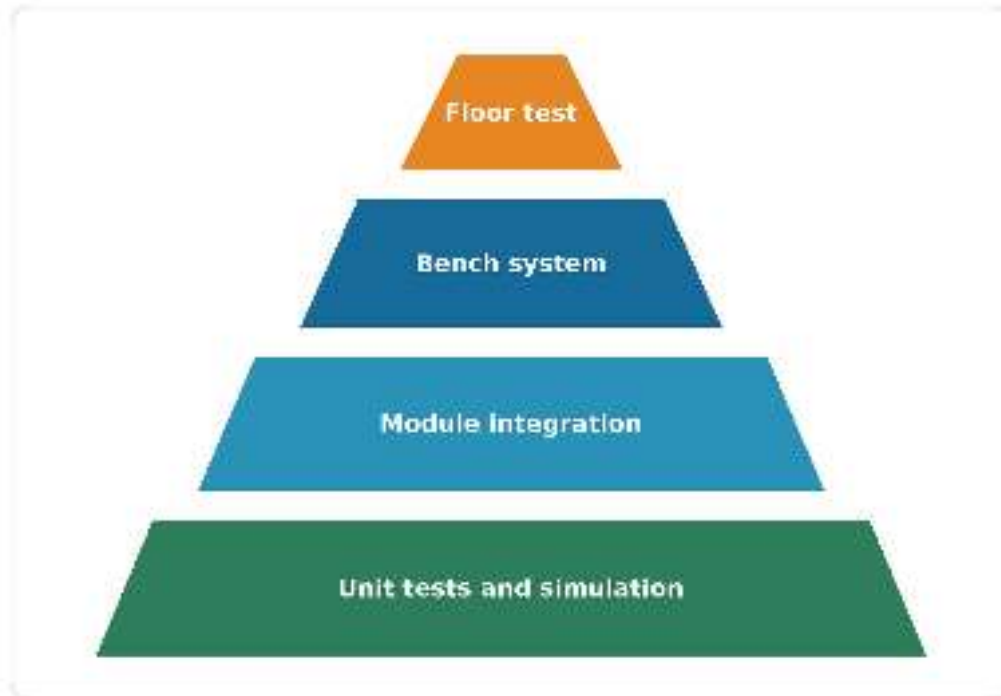


Figure 8 - Validation pyramid: many inexpensive tests support a few complete trials.
Source: Created by the author.

Tests begin with functions and modules, progress to integration, and end with the complete robot. The more physical and integrated a test is, the more expensive and dangerous it is to discover a defect. Exercise parser, calculations, and policy without hardware.

18.2 Minimum matrix

ID	Test	Condition	Expected result
PWR-01	Polarity	without boards	all outputs correct
PWR-02	Servo start	worst validated load	logic does not reset
PWR-03	Undervoltage	supply reduced in a controlled manner	actuators stop and fault is recorded
FW-01	Boot	any reset	DISARMED, motors off
FW-02	Long line	> buffer	error, no execution
FW-03	Lost heartbeat	during motion	STOP within limit
FW-04	Unknown command	valid line	refused
AG-01	Invented command	LLM response	blocked by catalog
AG-02	Out-of-range parameter	PWM 999	blocked by validator
AG-03	No confirmation	motion	not sent
NET-01	Network loss	during operation	local safety preserved
VIS-01	Missing camera	startup and runtime	clear error, no motion enabled

ID	Test	Condition	Expected result
VOI-01	Ambiguous phrase	noise	question/refusal, no action
RAG-01	Conflicting document	old revision	conflict reported

18.3 Electrical measurements

Record in each mode:

Mode	Battery V	Battery I	Minimum logic V	Peak servo I	Temperature
Idle					
Wheels raised					
Floor start					
Turn					
Arm, worst validated pose					

Use safe instruments and methods. A single reading does not represent a fast peak; an oscilloscope or recorder may be necessary for voltage drop.

18.4 Mechanical measurements

- total mass and distribution;
- actual wheel diameter;
- current and speed on level floor;
- maximum validated slope;
- stopping distance by speed;
- joint backlash;
- position repeatability;
- motor and servo temperature;
- stability with arms in extreme positions.

18.5 AI evaluation

Create a fixed set of English phrases, including variations and errors. Classify:

- response without action;
- sensor query;
- valid action;
- ambiguous action;
- prohibited action;
- prompt attack;
- impossible request;

- loss of context.

For each case, record proposed JSON, validator decision, confirmation requirement, and final response. Changing models requires repeating the set; models with similar names may behave differently.

18.6 Fault injection

Induce faults in a controlled way:

- remove the serial cable;
- stop the LLM server;
- delay a response;
- send corrupted bytes;
- block a sensor;
- simulate high current through a test input;
- restart a process;
- fill log space in a test environment;
- supply a RAG document with a malicious instruction.

The goal is to verify safe output and diagnosis, not to damage equipment.

18.7 Test record

Each test includes:

TEXT
 ID and title:
 Date and person responsible:
 Mechanical version:
 Firmware and commit:
 Application/TCHATGPT and commit:
 Hardware and configuration:
 Preconditions:
 Steps:
 Expected result:
 Observed result:
 Measurements and evidence:
 Conclusion: PASS / FAIL / BLOCKED
 Linked anomaly:

18.8 Commissioning criterion

The robot may operate under supervision when every critical test passes, there is no open safety fault, and the tested configuration matches the assembled one. One successful demonstration does not replace the test suite.

19. Maintenance and evolution

19.1 Periodic inspection

Before each session, inspect the battery, connectors, fuses, wheels, screws, cables, and emergency stop. After impact or transport, repeat the mechanical inspection and raised-wheel test.

Periodically:

- check pack capacity and balance using an appropriate procedure;
- clean and inspect motors and joints;
- look for heating and discoloration at connectors;
- review undervoltage, overcurrent, and watchdog logs;
- back up configuration and the RAG index;
- apply updates only with a rollback plan;
- repeat affected tests.

19.2 Compatibility

Maintain a matrix of operating system, architecture, Lazarus, FPC, packages, and runtime. "Compiles on Windows" does not prove Linux or ARM support. CI should build samples per package and a Robotinics-specific set. Hardware tests remain separate from compilation tests.

19.3 Reproducible repository

Rev. 3 recommends:

1. a Robotinics tag corresponding to the book;
2. a `docs/rev3` directory with an editable source for the book;
3. Rev. 3 firmware separate from legacy firmware;
4. a Lazarus sample named `robotinics_ai_controller`;
5. version-controlled schematics and BOM;
6. installation and verification script;
7. CI for firmware and Lazarus;
8. releases with checksums;
9. vulnerability and contribution policies.

Compiled files, objects, and caches must not replace sources. Removed historical tools may remain under a `legacy` tag with a security warning.

19.4 Technical roadmap

Priority 0 - safety and reproduction

- validate pack, BMS, charger, fuse, and rails;

- implement Rev. 3 firmware with watchdog;
- correct the pin map and commands;
- create a physical stop and fault test;
- freeze the BOM and versions.

Priority 1 - intelligent controller

- adapt `agent_serial_demo` to Robotinics;
- integrate `TAIAgentSafety` and a specific validator;
- create simulation mode;
- add telemetry and TraceID;
- test Windows/Linux x64.

Priority 2 - Raspberry Pi ARM64

- create CI or a reproducible ARM64 build;
- validate `openai_core`, serial, and agent;
- validate voice and vision separately;
- measure performance and temperature;
- promote components only with evidence.

Priority 3 - perception and knowledge

- RAG with versioned documents;
- voice with consent and confirmation;
- vision with an identified real backend;
- evaluation datasets;
- observability and regression testing.

19.5 Publication

Before publishing:

- define licenses for text and figures;
- preserve the GPLv3 license and notices applicable to TCHATGPT;
- audit the origin of third-party images and models;
- remove passwords, tokens, and personal data;
- verify links and commits;
- conduct technical reviews of the battery, electrical system, and mechanics;
- generate a PDF with bookmarks, links, and selectable text;
- provide DOCX or equivalent source for future revisions.

The Robotinics repository has no formal license in the examined snapshot. Any claim that the project is open should be accompanied by a license file chosen by the author. This edition does not choose a license on his behalf.

Appendix A - Suggested bill of materials

This list organizes categories. Final models and quantities depend on the assembly and should appear in the version-controlled BOM.

Subsystem	Items	Verification
Structure	STL/CAD parts, screws, inserts, mounts	material, revision, load, and clearance
Traction	geared motors, wheels, driver	voltage, stall current, continuous torque
Control	Arduino Mega or compatible board	pins, memory, interfaces
Computing	compatible Raspberry Pi, storage	architecture, power, temperature
Power	3S pack, BMS, charger, fuse, switch	chemistry, current, certification, and assembly
Conversion	traction, servo, and logic rails	voltage, current, ripple, and cooling
Servos	actuators and mounts	calculated torque, limits, and current
Sensors	ultrasound, voltage, current, IMU/gas as required	logic level, range, calibration
Interface	camera, microphone, speaker	driver, consent, consumption
Safety	emergency stop, guards, fastening	access, test, and safe failure
Wiring	wires, terminals, connectors, labels	current, drop, polarization, and flexing

Do not buy from the historical BOM without checking availability and specifications. Clones sold under the same commercial name may differ.

Appendix B - Command protocol

B.1 Rules

- Commands use restricted ASCII/UTF-8; one line per message.
- Termination is LF or CRLF.
- Reference configuration allows at most 127 bytes of content per line.
- The first token identifies the command exactly.
- Decimal numbers omit units only when the contract defines the unit.
- STOP, DISARM, PING, STATUS?, and MAN do not depend on AI.
- Motion requires the ARMED state.
- Responses include OK, ERR, STATE, or EVENT.

B.2 Minimum catalog

Command	Parameters	State	Effect
PING	none	any	responds OK PING
MAN	none	any	publishes catalog
STATUS?	none	any	returns state and faults
ARM	according to policy	DISARMED	runs self-test and arms
DISARM	none	any	disables actuators
STOP	none	any	stops motion
MOVEFWD	PWM, ms	ARMED	bounded forward motion
MOVEBACK	PWM, ms	ARMED	bounded reverse motion
TURNLEFT	PWM, ms	ARMED	bounded left turn
TURNRIGHT	PWM, ms	ARMED	bounded right turn
RANGE?	optional sensor	any	distance or timeout
BATTERY?	none	any	voltage and state
SERVO	axis, angle, speed	ARMED	pose within limits

B.3 Legacy aliases

Legacy	Rev. 3	Note
FRENTE	MOVEFWD	conservative default parameters
RE	MOVEBACK	conservative default parameters
GESQ	TURNLEFT	bounded duration
GDIR	TURNRIGHT	bounded duration
PARA	STOP	PARAR may be accepted only as a documented alias

Aliases do not bypass state, catalog, or limits.

Appendix C - Correction matrix for the previous edition

Approximate location	Identified problem	Correction adopted in Rev. 3
pp. 32-45	force in N compared directly with torque in N-m	separate formulas and conversion by radius
pp. 38-39	mass, weight, kgf, N, and slope mixed	$m \times g \times \sin(\theta)$ and dimensional example
pp. 42-45	arm torque in $\text{kgf} \cdot \text{cm}^2$ and inconsistent masses	sum of moments in N-m, centers of mass, and margin
pp. 49-50	$0.02 \text{ A} = 200 \text{ mA}$	$0.02 \text{ A} = 20 \text{ mA}$
pp. 53-56	apparent/reactive power applied to DC pack	calculation using W, Wh, and conversion efficiency
pp. 58-59	series-cell capacities added	3S remains 5.2 Ah; only voltage adds
p. 96	$20.75 \times 10^{-3} \text{ F}$ converted to 20.75 μF	correct value: 20,750 μF
pp. 97-113	lithium charging based on wrong current and indefinite connection	3S CC/CV charger, BMS, balancing, and datasheet
pp. 108-113	buck set for 12.6 V from a 12 V input	buck cannot raise voltage; architecture redesigned
p. 106	NO/NC inverted	NO open at rest; NC closed at rest
p. 113	ammeter connected in parallel with output	current measured in series; short-circuit warning
pp. 135-146	currents from different rails added and single supply undersized	power-based budget and separate rails
pp. 140-144	L298N described without PWM/as isolation	PWM on enables, no isolation, explicit losses
pp. 150-155	HC-SR04 described as 40 Hz	approximately 40 kHz
ultrasonic example	undefined <code>echoPin/trigPin</code> and same sample repeated	defined pins, timeout, and distinct samples
pp. 187-193	servo associated only with PWM pins	library uses timers and compatible digital pins
p. 196	$5/330 = 0.15 \text{ A}$	$5/330 \approx 0.015 \text{ A}$; formula uses LED drop
p. 216	MySQL example with typographic quotes and syntax errors	compilable examples and parameterized queries
pp. 218-220	C/C++ servers with unsafe buffers, formatting, and shell	removed from operational architecture; validated gateway
pp. 221-223	Motion confused with semantic vision	capture/motion separated from detection/classification
pp. 252-255	OpenCV 2.4 and old installation	current documentation and TCHATGPT backends
<code>face.py</code>	VPython animation presented as recognition	correctly classified; use a real detector
pp. 256-267	old PocketSphinx as the only route	modular Whisper/STT, with Sphinx retained only as legacy
pp. 268-275	FANN and absent/incomplete examples	current ML components or explicitly historical documentation
pp. 280-297	PHP5, MySQL root, exposed phpMyAdmin, <code>chmod 777</code>	least privilege, local service, authentication, and permissions
firmware	<code>String</code> , delays, shared buffer, and no timeout	fixed parser, modules, watchdog, and limits
firmware	invalid GPS macros and overflow/blocking risk	correct definitions, bounded buffer, and timeout
firmware	<code>ULTRA</code> tested before <code>ULTRA1/2</code>	exact comparison of the first token
firmware	<code>substring(5)</code> for prefix <code>SERIAL:</code>	token parser and actual length
tools	conflicting ports 7091/8091 and <code>PARAR/PARA</code>	central contract and documented aliases
repository	cited tools removed from current branch	declared snapshot/history; proposed new sample

Approximate location	Identified problem	Correction adopted in Rev. 3
editorial	missing chapter 6.2, non-clickable links, no bookmarks	continuous numbering, navigation, and links
PDF	open-project claim without formal license	license decision required before publication

Appendix D - Reference formulas

Quantity	Formula	Unit
Weight	$F = m \times g$	N
Slope force	$F = m \times g \times \sin(\theta)$	N
Rolling	$F = C_{rr} \times m \times g \times \cos(\theta)$	N
Acceleration	$F = m \times a$	N
Torque	$T = F \times r$	N·m
Arm torque	$T = \sum(m_i \times g \times d_i)$	N·m
DC power	$P = V \times I$	W
Energy	$E = P \times t$ or $V \times Ah$	Wh
LED resistor	$R = (V_{supply} - V_{LED}) / I$	Ω
Resistor power	$P = I^2 \times R$	W
Ripple capacitor	$C \approx I / (f \times \Delta V)$	F
Runtime	$t \approx E_{usable} / P_{average}$	h
Ultrasonic distance	$d = v_{sound} \times time / 2$	m

Use consistent units. Convert millimeters to meters and milliamperes to amperes before calculating. State assumptions and rounding.

Appendix E - Platform and dependency matrix

Layer	Windows x64	Linux x64	Raspberry ARM64	Note
openai_core	supported	supported	likely/experimental	validate HTTPS and runtime
serial/input	supported by samples	supported by samples	likely/experimental	permissions and device
serial agent	compiles in report	compiles according to environment	experimental	test hardware and GUI
voice	depends on executables/models	depends on executables/models	experimental	CPU/RAM cost
OpenCV Python	depends on Python/OpenCV	depends on Python/OpenCV	experimental	use <code>TAIPythonRuntime</code>
OpenCV native	partial loading	partial loading	experimental	incomplete native processing
RAG	compiles through sample	expected	experimental	measure memory and index

Do not promote "likely" to "supported" without a repeatable build and test.

Appendix F - References

Projects

1. Marcelo Maurin Martins. **Robotinics**. [Robotinics repository](#). Editorial snapshot 61d2a19d10e6701e27e46181e1a6c740b8ee4e30.
2. Marcelo Maurin Martins. **TCHATGPT - AI Component Suite for Lazarus / Free Pascal**. [TCHATGPT repository](#). Editorial snapshot 15d5e1a7780088701716896cfe9fb3afe0e7b71a.
3. TCHATGPT. [Platform compatibility](#).
4. TCHATGPT. [Agent Serial Demo](#).
5. TCHATGPT. [LLM and provider architecture](#).

Official technical documentation

1. Raspberry Pi Ltd. [Getting started - Raspberry Pi Documentation](#).
2. Arduino. [Arduino IDE 2 - Getting Started](#).
3. OpenCV. [Installation in Linux](#).
4. Texas Instruments. [Cell balancing buys extra run time and battery life](#).
5. Texas Instruments. [Precise Constant Current Regulation Helps Advance Fast-charging](#).
6. Fluke. [Digital multimeter safety and measurement guidance](#).
7. Omron. [Explanation of relay terms](#).

Appendix G - Glossary

Term	Definition
ADC	analog-to-digital converter
BMS	battery management/protection system
CC/CV	constant-current and constant-voltage charging
CI	continuous integration; automated build/test
Firmware	software executed by the microcontroller
GPIO	general-purpose input/output
Heartbeat	periodic presence/health message
LLM	large language model
PWM	pulse-width modulation
RAG	retrieval-augmented generation using documents
Stall	motor condition with a locked shaft
STT	speech-to-text conversion
TTS	text-to-speech conversion
Watchdog	timer that detects a hang or missing update

Closing remarks

Robotinics remains more valuable as an open learning platform than as a closed product. The third edition does not erase its history: it turns errors, limitations, and technological changes into part of the learning process.

The central principle is simple: artificial intelligence expands the ability to converse, perceive, and plan, but robot safety remains deterministic, local, measurable, and testable. With this separation, the project can evolve without confusing creativity with authorization.

The next step belongs to the community: reproduce, measure, record, and improve, always preserving origin and the ability to verify every decision.